

Week 6

Quantum Computing

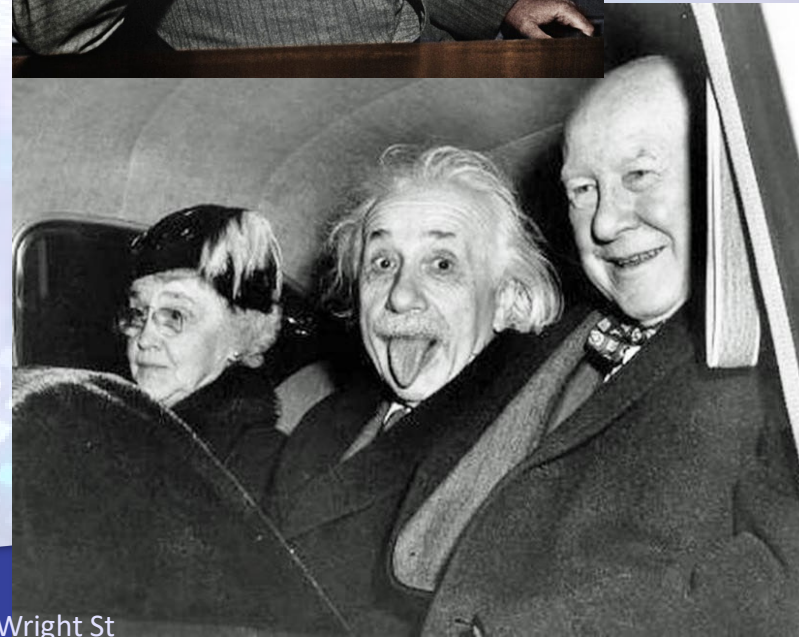
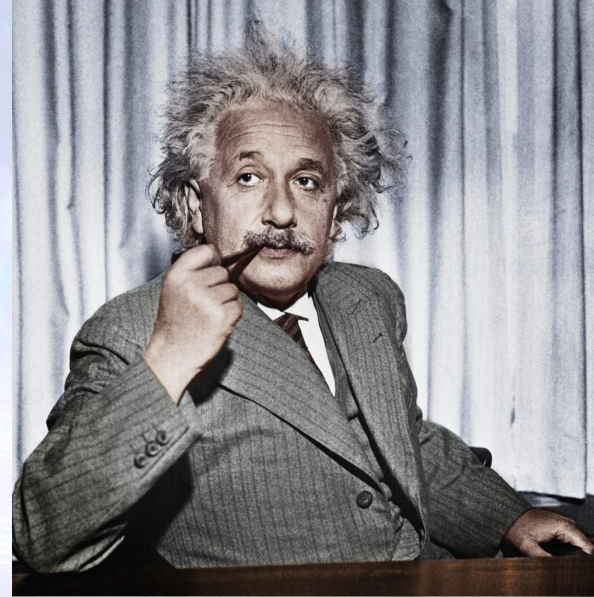
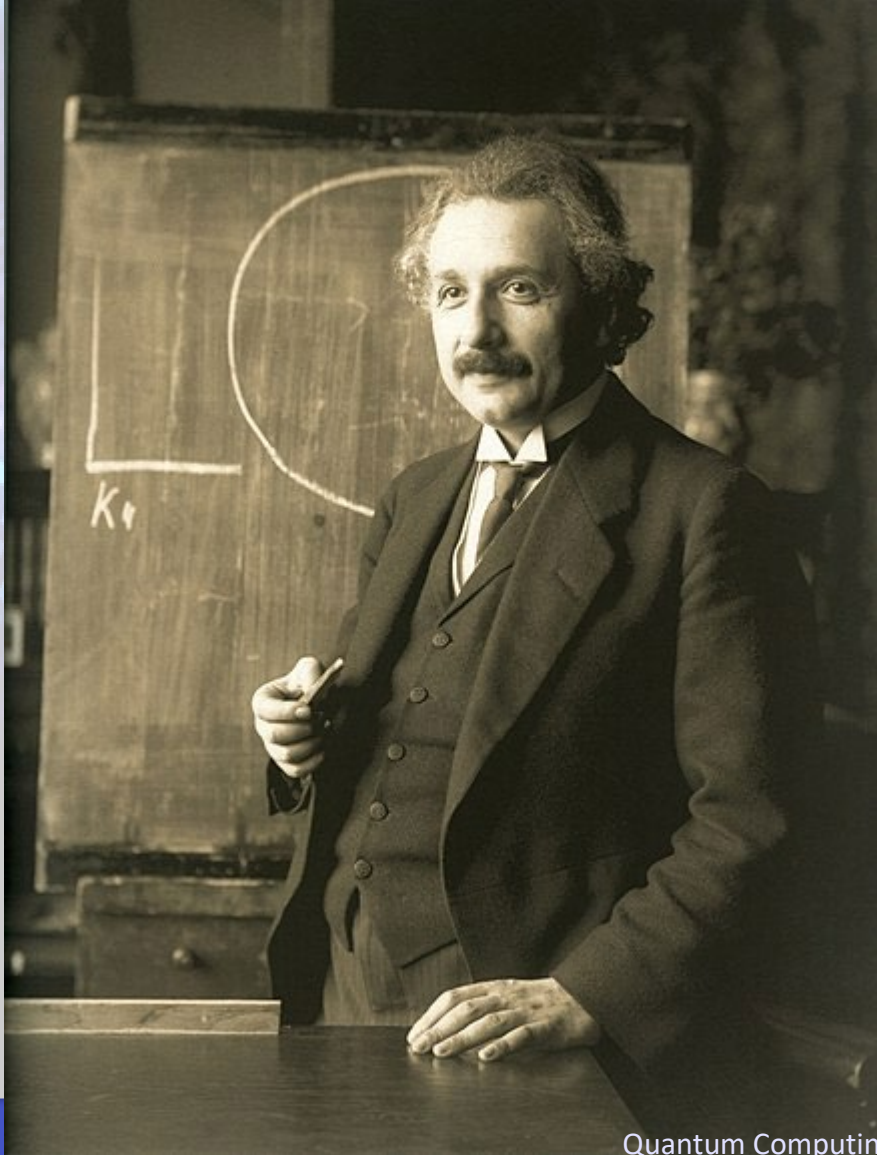
Midterm 1 - Review

Nella Ludlow, PhD
Wright State University

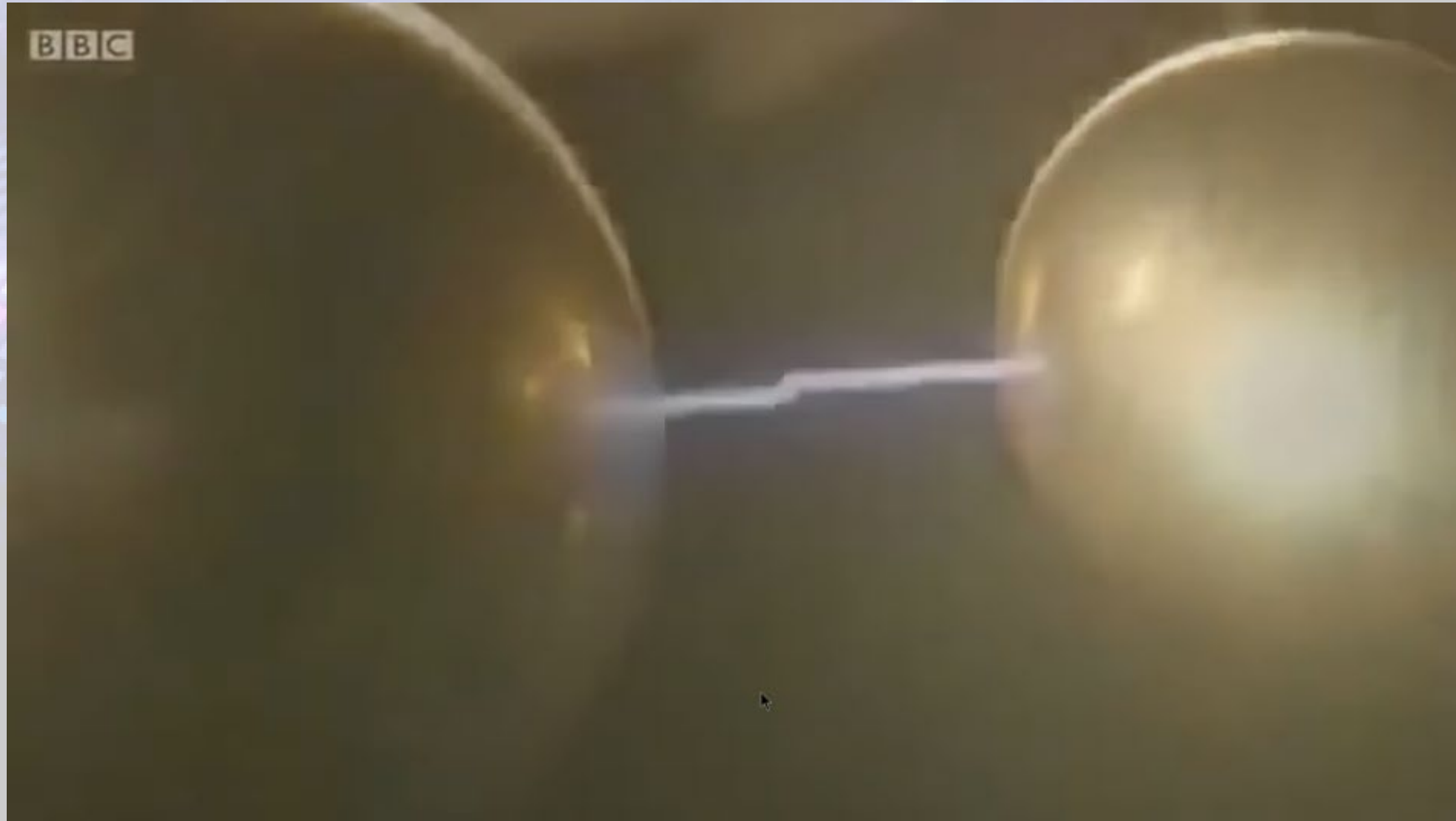
REVIEW - Week 1-2

General Quantum Computing

LOOK AT PHYSICS



LIGHT AS A WAVE



LIGHT

AS A PARTICLE



LIGHT

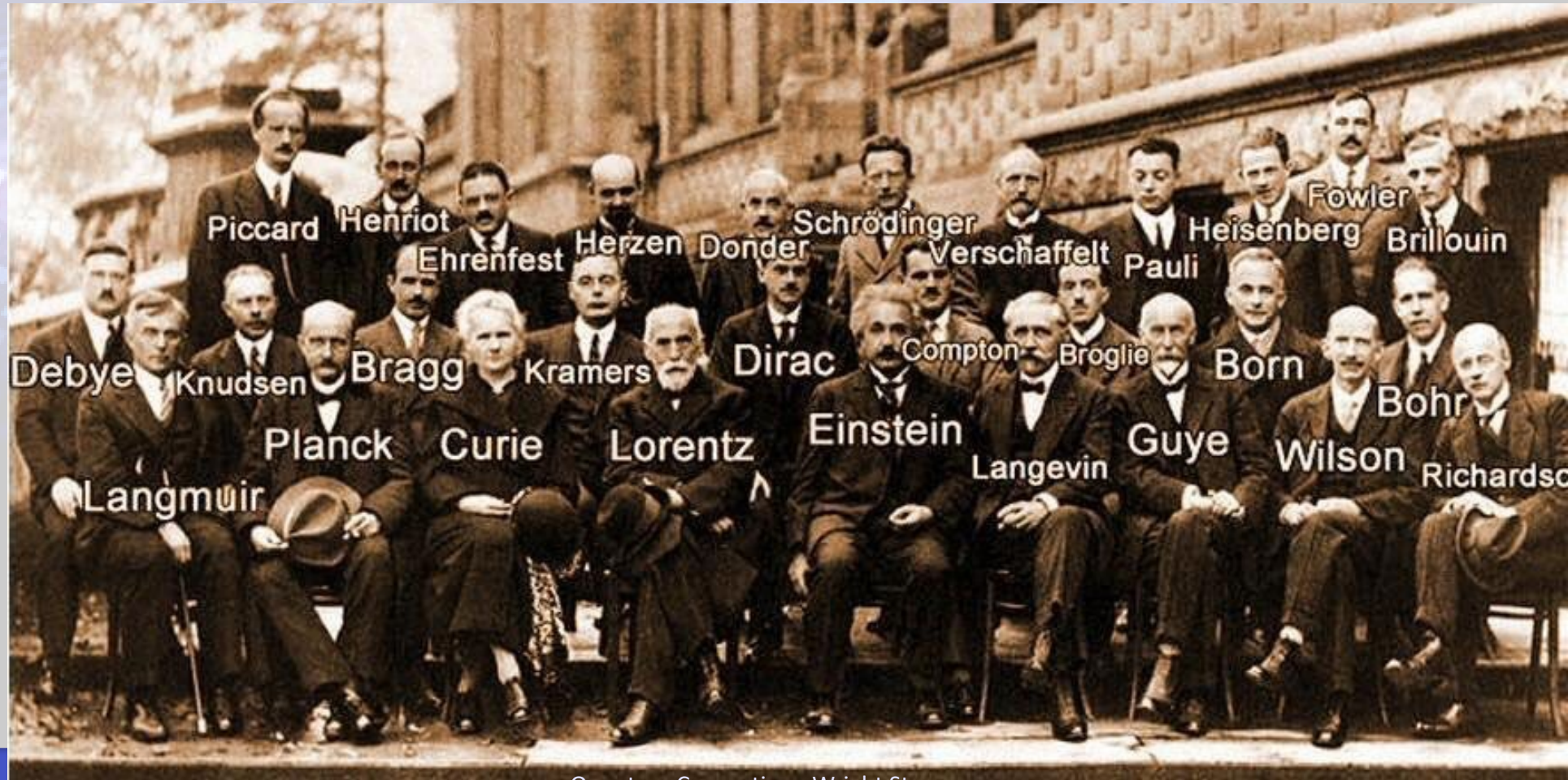
PARTICLE AND WAVE



QUANTUM CHANCE



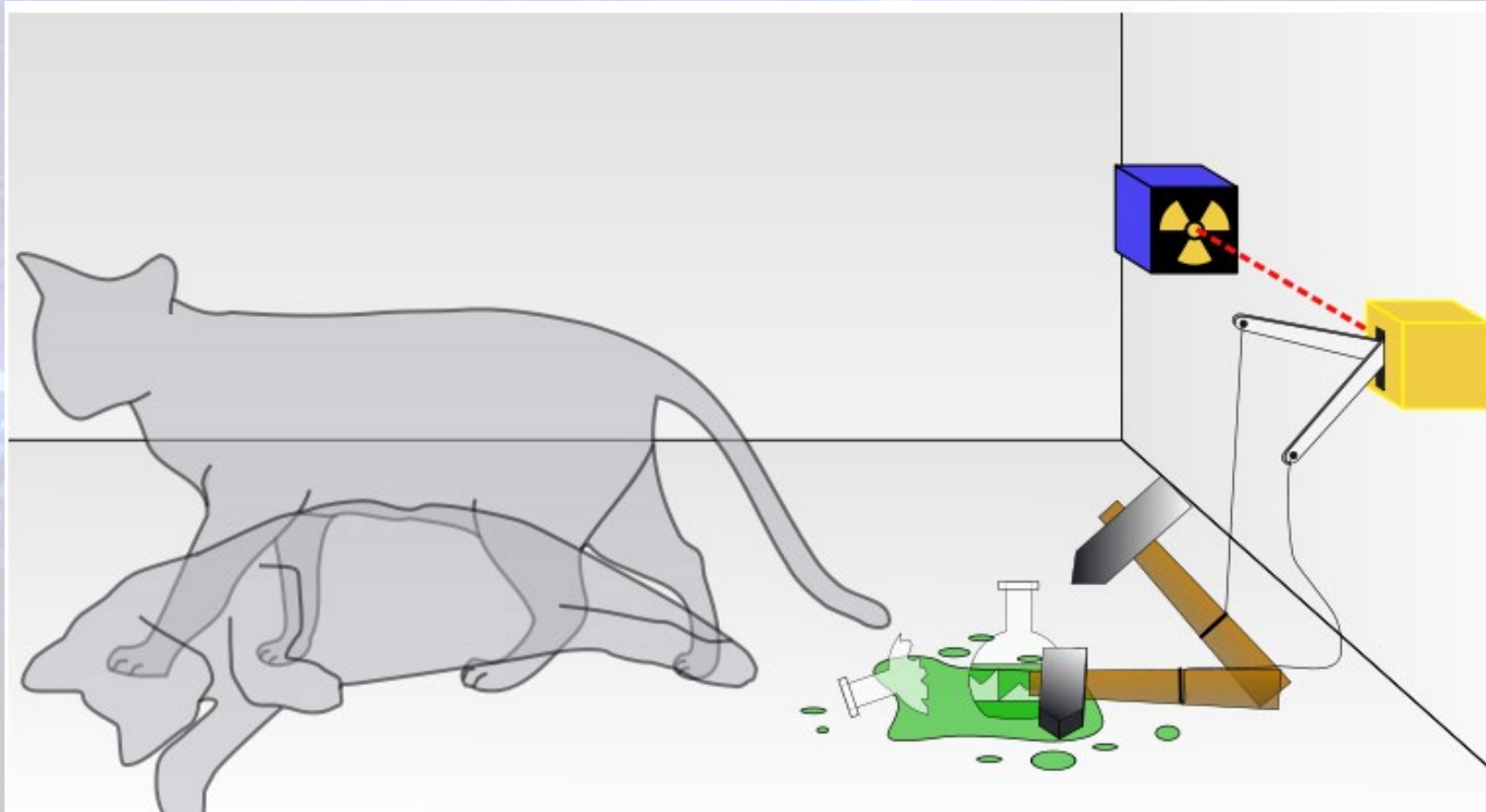
BOHR V EINSTEIN COPENHAGEN INTERPRETATION



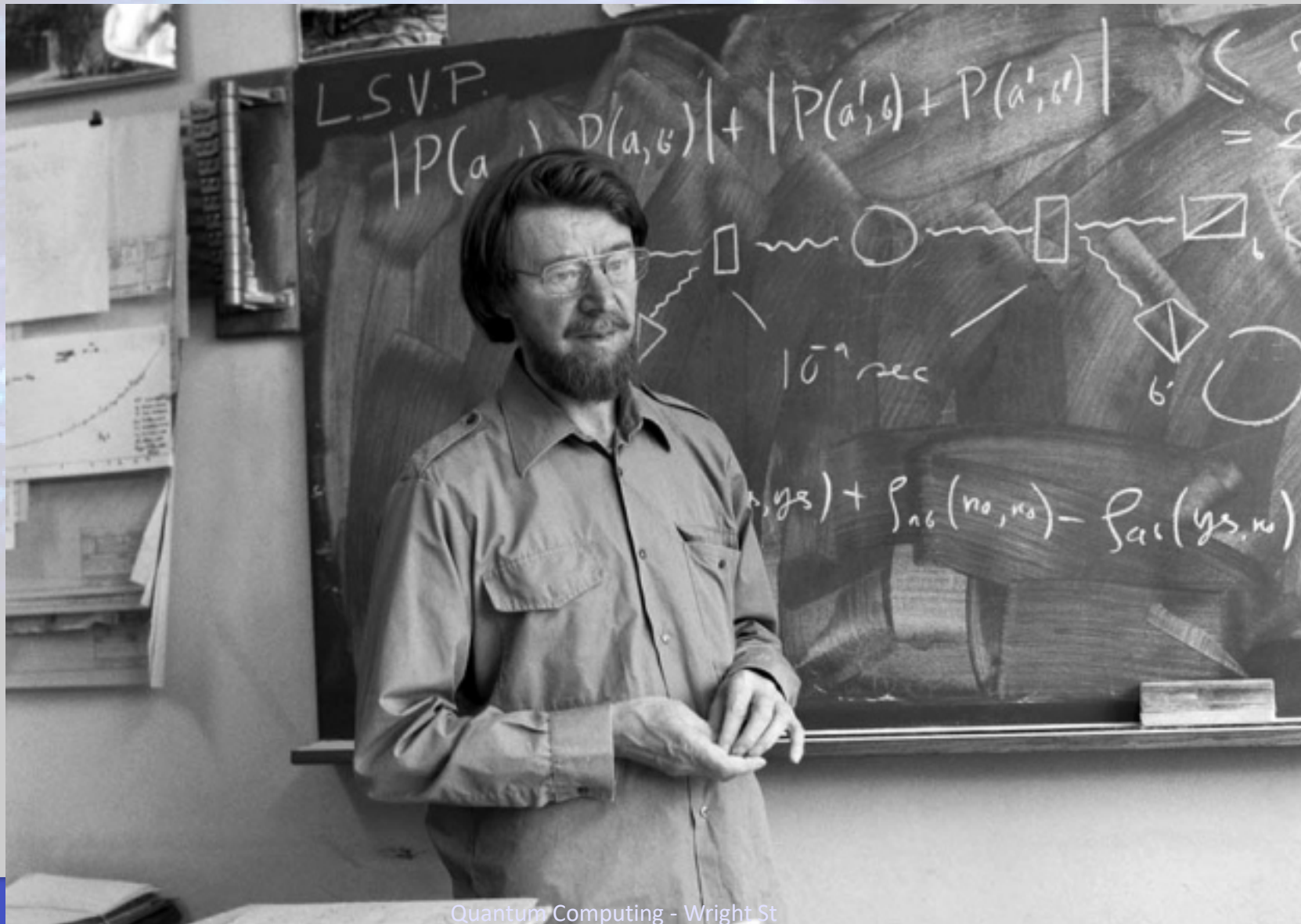
ENTANGLEMENT



SCHRODINGER'S CAT

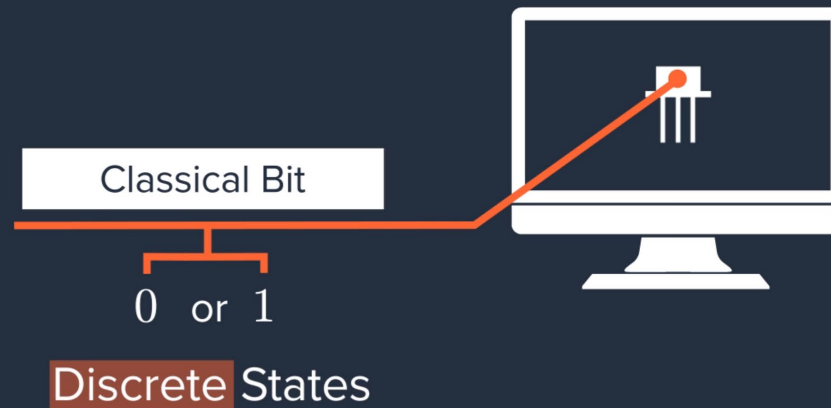


BELL'S INEQUALITY



Classical BIT

BIT = Binary Digit



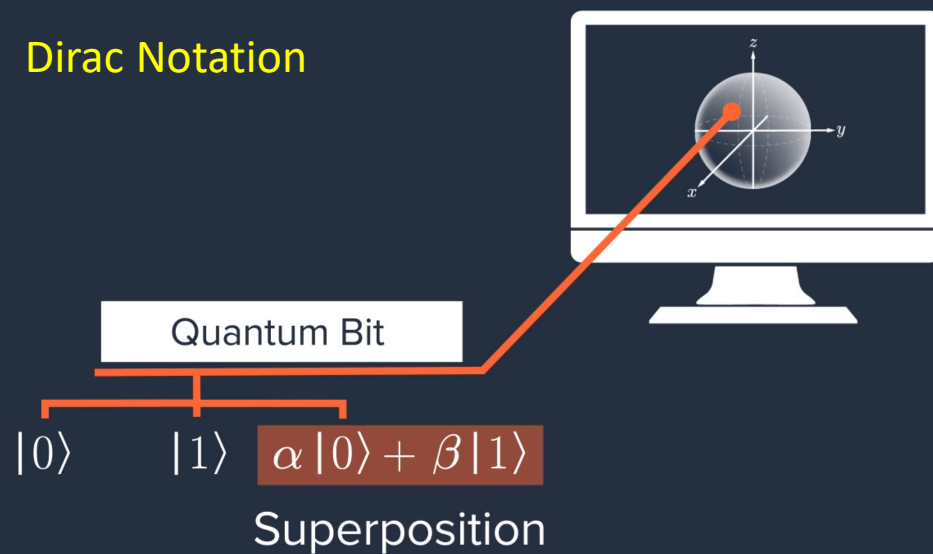
QUBIT

QUBIT = Quantum BIT



Not only represent 0 or 1
But also all positions in between

Dirac Notation



BLOCH SPHERE

MIT xPRO

Course | Discussion

Course > Week 1

< Previous

Ludlow

Next >

Think of it like the Earth

with 0 at the North Pole
and 1 at the South Pole

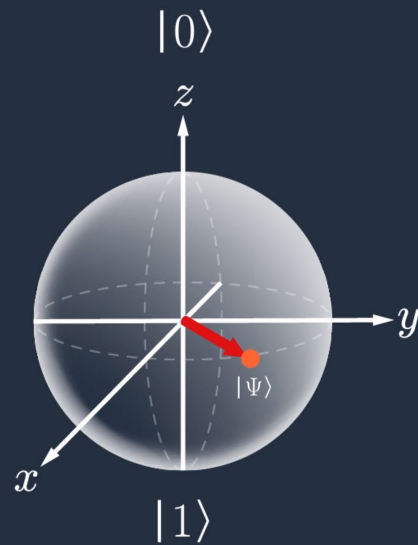
There are infinite number
of points on the surface of
the earth

0



1

SUPERPOSITION



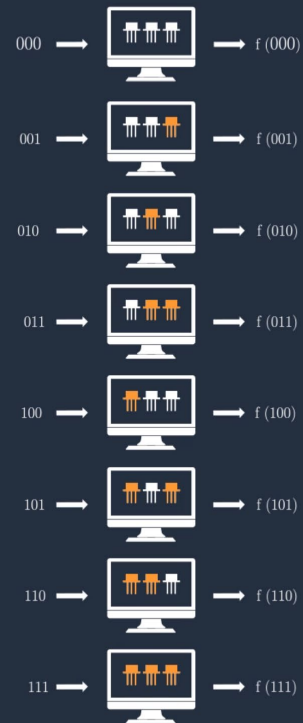
Superposition

$$|\Psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

TWO CHOICES: PARALLEL

If we want to try
DIFFERENT INPUT
States we only have
TWO choices:
MORE HARDWARE

Hardware



TWO CHOICES: SEQUENTIAL TIME

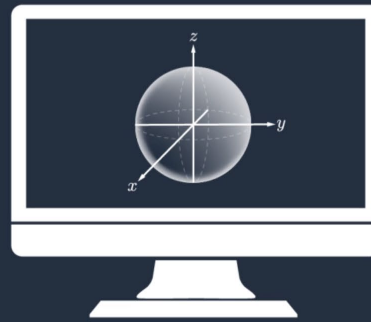
OR TAKE MORE TIME.....



Time

QUANTUM COMPUTING

Using Superposition
Can represent
simultaneously
ALL this different states



N Qubits : 2^N components in one state

QUANTUM STATE

MIT xPRO

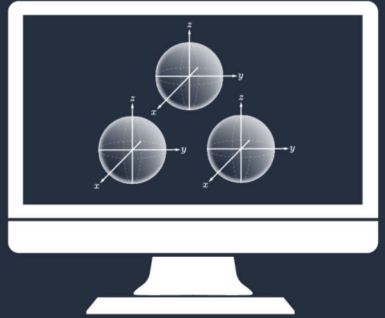
Course > Dis

Course > We

< Previous

Ludlow

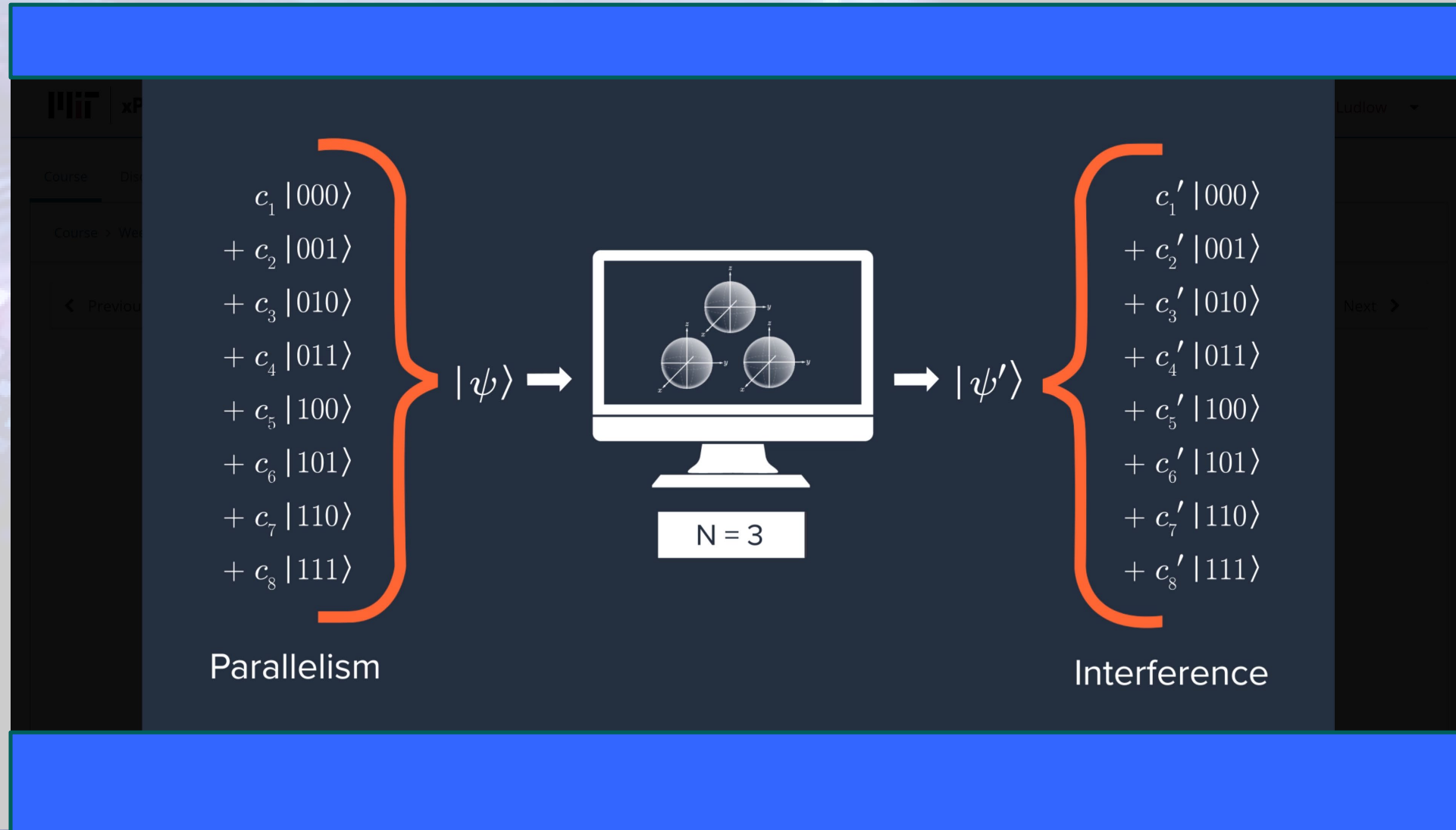
Next >

$$\begin{aligned} &c_1 |000\rangle \\ &+ c_2 |001\rangle \\ &+ c_3 |010\rangle \\ &+ c_4 |011\rangle \\ &+ c_5 |100\rangle \\ &+ c_6 |101\rangle \\ &+ c_7 |110\rangle \\ &+ c_8 |111\rangle \end{aligned} \Bigg\} |\psi\rangle$$


N = 3

QUANTUM

PARALLELISM & INTERFERENCE

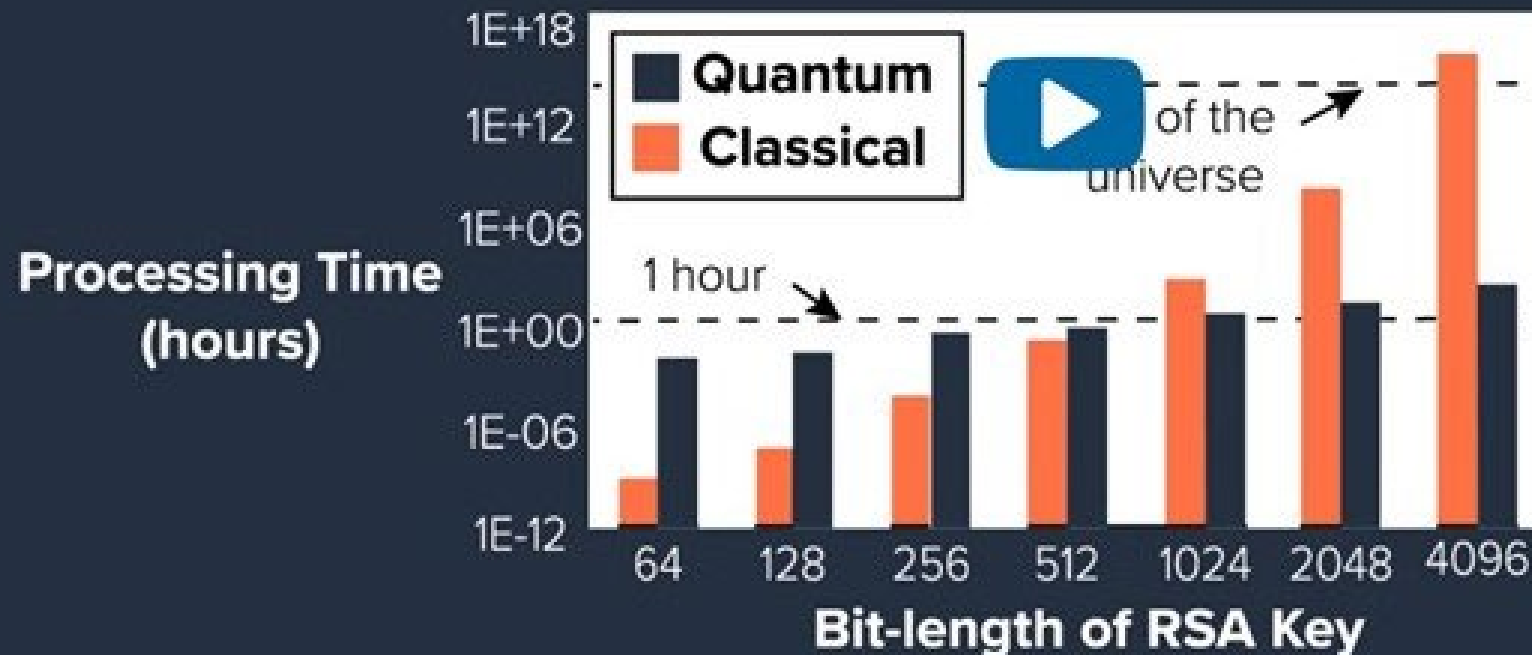


CODE BREAKING

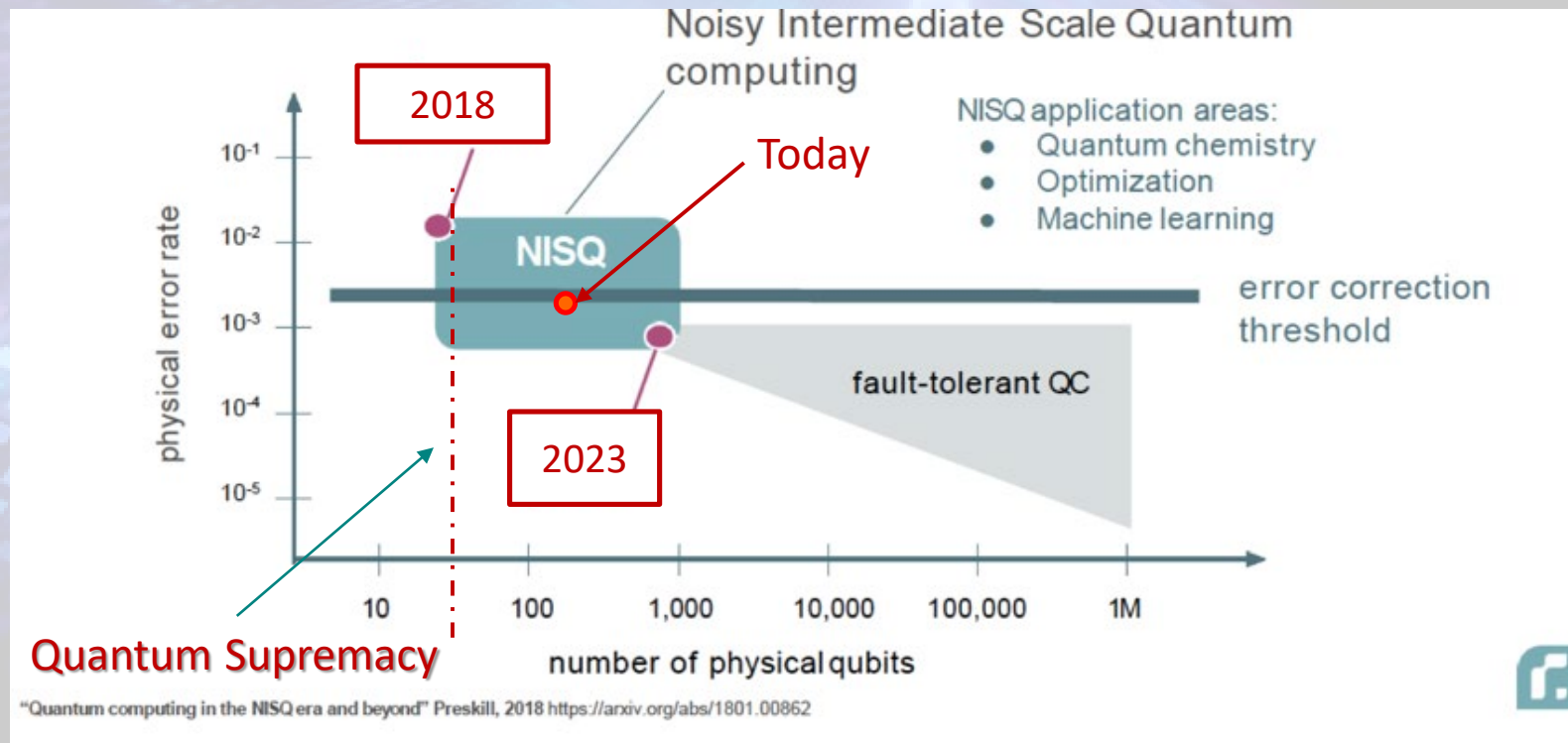


Shor's Algorithm

Shor's Algorithm for Prime Factorization: RSA Key Decryption



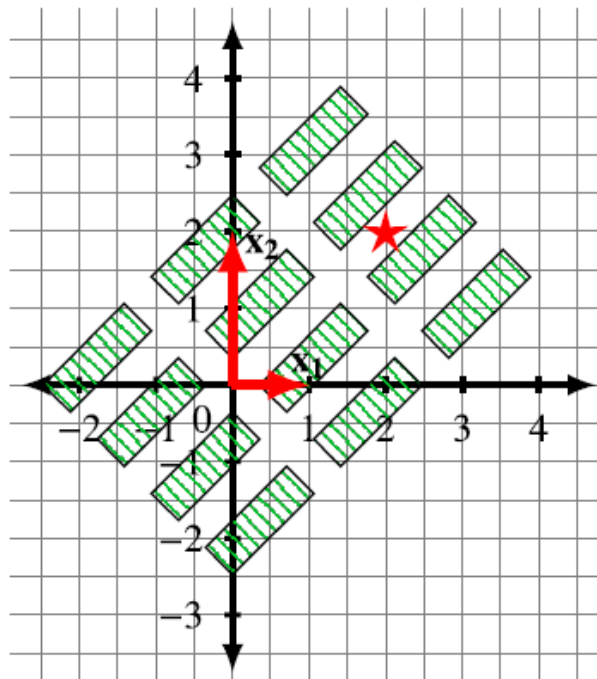
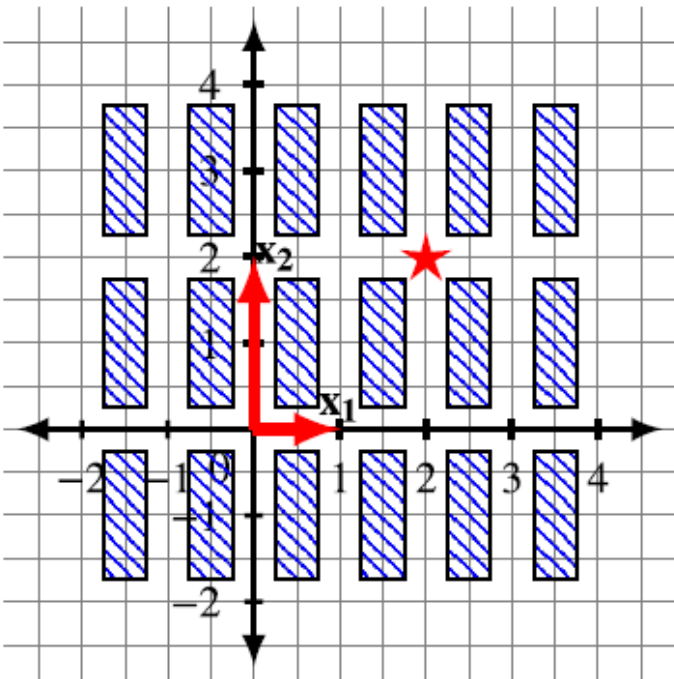
NISQ AND Q SUPREMACY



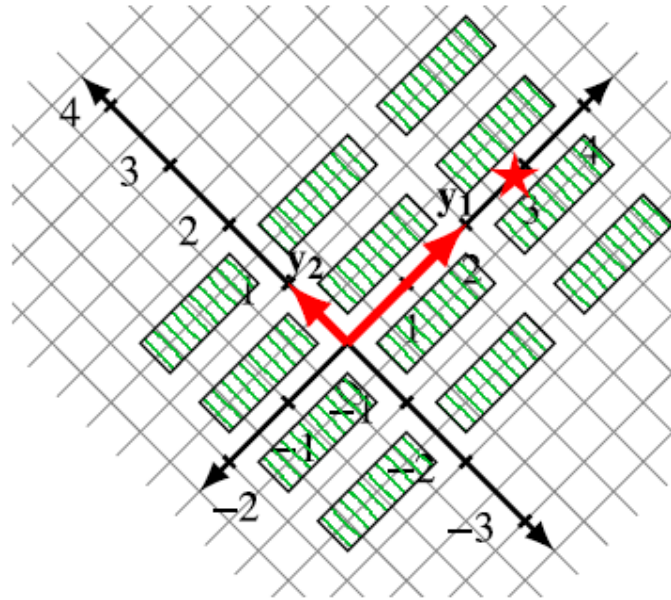
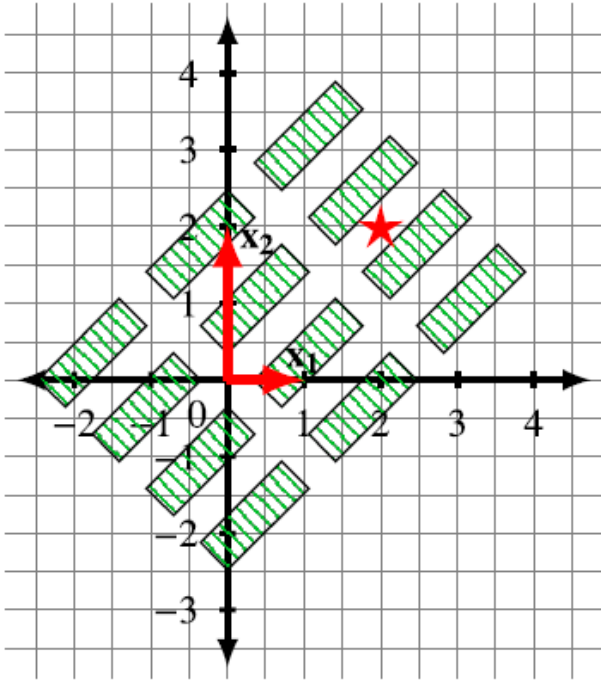
REVIEW - Week 3

Linear Algebra

Why Matrices?



Create a New Coordinate System?

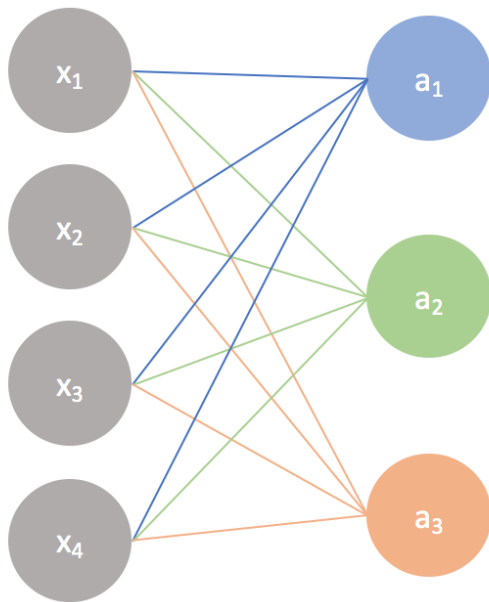


Neural Network is Represented by a Matrix

Input layer

Output layer

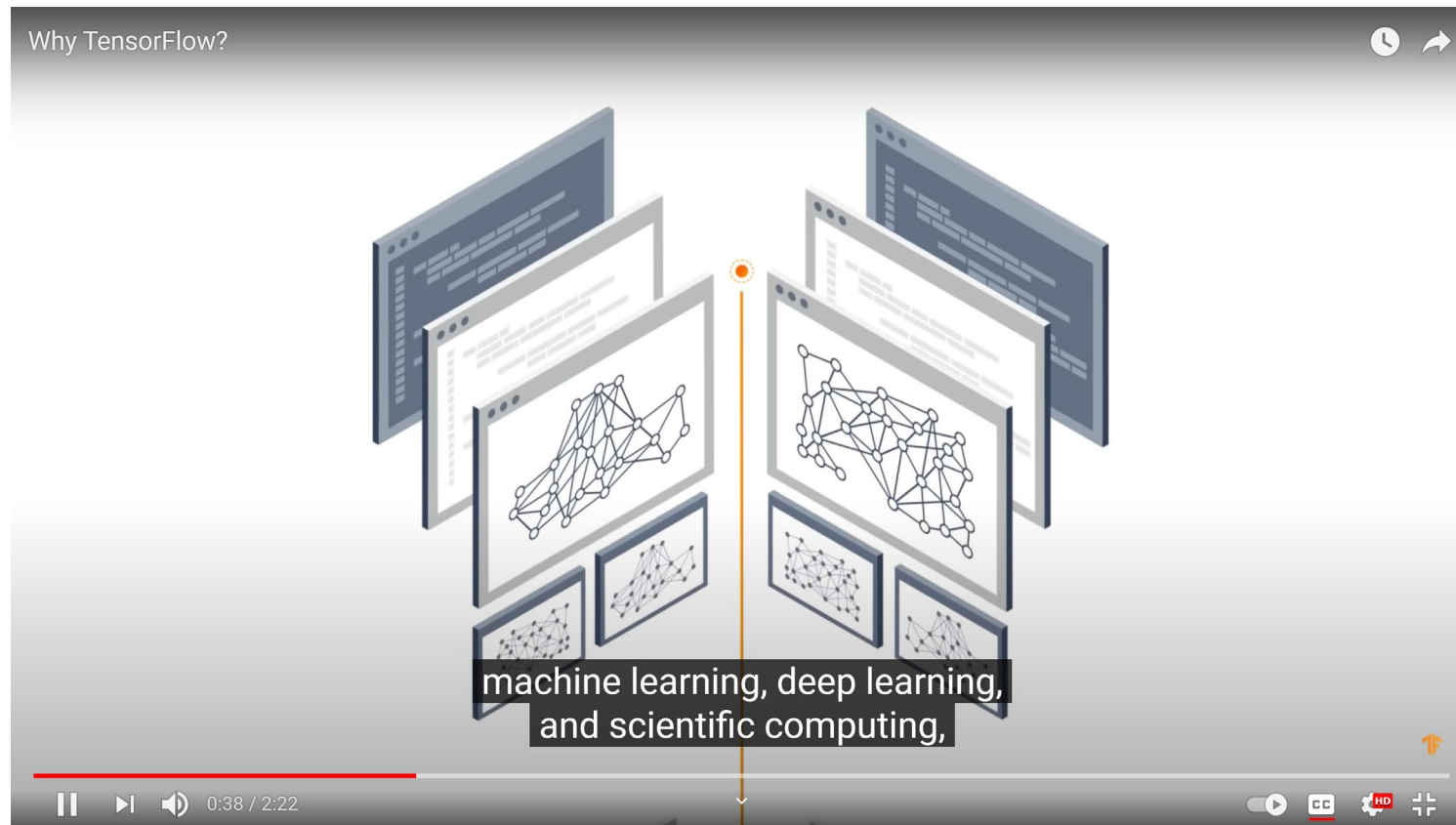
A simple neural network



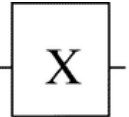
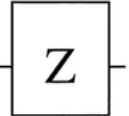
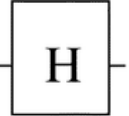
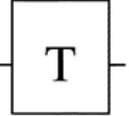
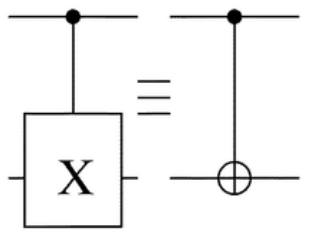
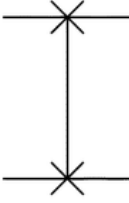
$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow{\text{activation}} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

TensorFlow

<https://www.youtube.com/watch?v=yjprp0oH5c8>



Quantum may be the ultimate Matrix Multiplication / Tensor Machine

X Gate Bit-flip, Not		$\equiv$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \beta 0\rangle + \alpha 1\rangle$
Z Gate Phase-flip		$\equiv$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \alpha 0\rangle - \beta 1\rangle$
H Gate Hadamard		$\equiv \frac{1}{\sqrt{2}}$	$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \frac{\alpha+\beta 0\rangle + \alpha-\beta 1\rangle}{\sqrt{2}}$
T Gate		$\equiv$	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \alpha 0\rangle + e^{i\pi/4}\beta 1\rangle$
Controlled Not Controlled X CNot		$\equiv$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = a 00\rangle + b 01\rangle + d 10\rangle + c 11\rangle$
Swap		$\equiv$	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = a 00\rangle + c 01\rangle + b 10\rangle + d 11\rangle$

Matrix Addition and Scalar Multiplication

- ▶ You can add two matrices (of the same dimension) by adding their corresponding entries.

Definition of Matrix Addition

If $A = [a_{ij}]$ and $B = [b_{ij}]$ are matrices of dimension $m \times n$, then their sum is the $m \times n$ matrix given by

$$A + B = [a_{ij} + b_{ij}].$$

The sum of two matrices of different dimensions is undefined.

Matrix Addition - Steps

Steps

$$\begin{pmatrix} 1 & 5 & -1 \\ 11 & 3 & 4 \\ 1 & -1 & 3 \end{pmatrix} + \begin{pmatrix} 7 & 2 & 9 \\ 1 & 2 & 5 \\ 0 & 2 & 4 \end{pmatrix}$$

Add the elements in the matching positions

$$= \begin{pmatrix} 1+7 & 5+2 & (-1)+9 \\ 11+1 & 3+2 & 4+5 \\ 1+0 & (-1)+2 & 3+4 \end{pmatrix}$$

Simplify each element

$$= \begin{pmatrix} 8 & 7 & 8 \\ 12 & 5 & 9 \\ 1 & 1 & 7 \end{pmatrix}$$

Matrice Scalar Multiplication - Steps

$$3 \begin{pmatrix} 1 & 3 & 8 \\ 2 & -3 & 4 \\ 5 & 6 & 7 \end{pmatrix} = \begin{pmatrix} 3 & 9 & 24 \\ 6 & -9 & 12 \\ 15 & 18 & 21 \end{pmatrix}$$

Steps

$$3 \begin{pmatrix} 1 & 3 & 8 \\ 2 & -3 & 4 \\ 5 & 6 & 7 \end{pmatrix}$$

Scalar multiplication: Multiply each of the matrix elements by a scalar

$$= \begin{pmatrix} 3 \cdot 1 & 3 \cdot 3 & 3 \cdot 8 \\ 3 \cdot 2 & 3(-3) & 3 \cdot 4 \\ 3 \cdot 5 & 3 \cdot 6 & 3 \cdot 7 \end{pmatrix}$$

Simplify each element

$$= \begin{pmatrix} 3 & 9 & 24 \\ 6 & -9 & 12 \\ 15 & 18 & 21 \end{pmatrix}$$

Matrix Addition and Scalar Multiplication

Properties of Matrix Addition and Scalar Multiplication

Let A , B , and C be $m \times n$ matrices and let c and d be scalars.

1. $A + B = B + A$

Commutative Property of Matrix Addition

2. $A + (B + C) = (A + B) + C$

Associative Property of Matrix Addition

3. $(cd)A = c(dA)$

Associative Property of Scalar Multiplication

4. $1A = A$

Scalar Identity

5. $A + O = A$

Additive Identity

6. $c(A + B) = cA + cB$

Distributive Property

7. $(c + d)A = cA + dA$

Distributive Property

Example 6 - *Solving a Matrix Equation*

► Solve for X in the equation

►

$$3X + A = B$$

► where

► $A = \begin{bmatrix} 1 & -2 \\ 0 & 3 \end{bmatrix}$ and $B = \begin{bmatrix} -3 & 4 \\ 2 & 1 \end{bmatrix}$.

Example 6 - *Solution*

► Begin by solving the equation for X to obtain

► $3X = B - A$

► $X = \frac{1}{3}(B - A)$

► Now, using the A matrices and B , you have

►
$$X = \frac{1}{3} \left(\begin{bmatrix} -3 & 4 \\ 2 & 1 \end{bmatrix} - \begin{bmatrix} 1 & -2 \\ 0 & 3 \end{bmatrix} \right)$$

$$= \frac{1}{3} \begin{bmatrix} -4 & 6 \\ 2 & -2 \end{bmatrix}$$

$$= \begin{bmatrix} -\frac{4}{3} & 2 \\ \frac{2}{3} & -\frac{2}{3} \end{bmatrix}.$$

Substitute the matrices

Subtract matrix A from matrix B

Multiply the resulting matrix by $\frac{1}{3}$

Matrix Multiplication

- ▶ Another basic matrix operation is **matrix multiplication**. You will see later, that this definition of the product of two matrices has many practical applications.

Definition of Matrix Multiplication

If $A = [a_{ij}]$ is an $m \times n$ matrix and $B = [b_{ij}]$ is an $n \times p$ matrix, then the product AB is an $m \times p$ matrix given by

$$AB = [c_{ij}]$$

where $c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + a_{i3}b_{3j} + \cdots + a_{in}b_{nj}$.

Example 7 - Finding the Product of Two Matrices

► Find the product AB using $A = \begin{bmatrix} -1 & 3 \\ 4 & -2 \\ 5 & 0 \end{bmatrix}$ and $B = \begin{bmatrix} -3 & 2 \\ -4 & 1 \end{bmatrix}$.

► Solution:

►

First, note that the product is defined because the number of columns of A is equal to the number of rows of B . Moreover, the product AB has dimension 3×2 .

To find the entries of the product, multiply each row of A by each column of B .

Matrix Multiplication

- The general pattern for matrix multiplication is as follows.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1j} & \dots & b_{1p} \\ b_{21} & b_{22} & \dots & b_{2j} & \dots & b_{2p} \\ b_{31} & b_{32} & \dots & b_{3j} & \dots & b_{3p} \\ \vdots & \vdots & & \vdots & & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nj} & \dots & b_{np} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1j} & \dots & c_{1p} \\ c_{21} & c_{22} & \dots & c_{2j} & \dots & c_{2p} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{i1} & c_{i2} & \dots & c_{ij} & \dots & c_{ip} \\ \vdots & \vdots & & \vdots & & \vdots \\ c_{m1} & c_{m2} & \dots & c_{mj} & \dots & c_{mp} \end{bmatrix}$$

$a_{i1}b_{1j} + a_{i2}b_{2j} + a_{i3}b_{3j} + \dots + a_{in}b_{nj} = c_{ij}$

Matrice Multiplication - Steps

$$\begin{pmatrix} 11 & 3 \\ 7 & 11 \end{pmatrix} \begin{pmatrix} 8 & 0 & 1 \\ 2 & 3 & 5 \end{pmatrix} = \begin{pmatrix} 94 & 9 & 26 \\ 78 & 33 & 62 \end{pmatrix}$$

Steps

$$\begin{pmatrix} 11 & 3 \\ 7 & 11 \end{pmatrix} \begin{pmatrix} 8 & 0 & 1 \\ 2 & 3 & 5 \end{pmatrix}$$

Multiply the rows of the first matrix by the columns of the second matrix

$$= \begin{pmatrix} 11 \cdot 8 + 3 \cdot 2 & 11 \cdot 0 + 3 \cdot 3 & 11 \cdot 1 + 3 \cdot 5 \\ 7 \cdot 8 + 11 \cdot 2 & 7 \cdot 0 + 11 \cdot 3 & 7 \cdot 1 + 11 \cdot 5 \end{pmatrix}$$

Simplify each element

$$= \begin{pmatrix} 94 & 9 & 26 \\ 78 & 33 & 62 \end{pmatrix}$$

Identity Matrix - Acts like “x 1”

Definition of Identity Matrix

The $n \times n$ matrix that consists of 1's on its main diagonal and 0's elsewhere is called the **identity matrix of dimension $n \times n$** and is denoted by

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}. \quad \text{Identity matrix}$$

Note that an identity matrix must be *square*. When the dimension is understood to be $n \times n$, you can denote I_n simply by I .

Matrix Multiplication

- ▶ If A is an $n \times n$ matrix, then the identity matrix has the property that $AI_n = A$ and $I_n A = A$.
- ▶ For example,

$$\begin{bmatrix} 3 & -2 & 5 \\ 1 & 0 & 4 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & -2 & 5 \\ 1 & 0 & 4 \\ -1 & 2 & -3 \end{bmatrix} \quad AI = A$$

Algorithm Complexity - Matrix Multiplication

Where are we on this scale?

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	second	minute	hour	day	month	year	century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							

Fastest!

Fast!

Linear!

Merge Sort->

Most Sorts->

Slow!

Super slow!

Intractable!



The traditional way:

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

use three for-loop

$$\therefore T(n) = O(n^3)$$

The Divide-and-Conquer way:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = \underline{A_{11} \cdot B_{11}} + \underline{A_{12} \cdot B_{21}}$$

$$C_{12} = \underline{A_{11} \cdot B_{12}} + \underline{A_{12} \cdot B_{22}}$$

$$C_{21} = \underline{A_{21} \cdot B_{11}} + \underline{A_{22} \cdot B_{21}}$$

$$C_{22} = \underline{A_{21} \cdot B_{12}} + \underline{A_{22} \cdot B_{22}}$$

transform the problem of multiplying A and B , each of size $[n \times n]$
into 8 subproblems, each of size $\left[\frac{n}{2} \times \frac{n}{2}\right]$

Strassen Algorithm

► <https://www.youtube.com/watch?v=OSelhO6Qnlc>

Strassen algorithm for matrix multiplication (divide and conquer) - Inside code

$ae + bg = (a + d)(e + h) + a(g - e) - (a + b)h + (b - d)(g + h)$

$af + bh = a(f - h) + (a + b)h$

$ce + dg = (c + d)e + d(g - e)$

$cf + dh = a(f - h) + (a + d)(e + h) - (c + d)e - (a - c)(e + f)$

$c_{11} = ae + bg = p_1 + p_2 - p_3 + p_4$

$c_{12} = af + bh = p_5 + p_3$

$c_{21} = ce + dg = p_6 + p_2$

$c_{22} = cf + dh = p_5 + p_1 - p_6 - p_7$

$p_1 = (a + d)(e + h)$ $p_5 = a(f - h)$
 $p_2 = d(g - e)$ $p_6 = (c + d)e$
 $p_3 = (a + b)h$ $p_7 = (a - c)(e + f)$
 $p_4 = (b - d)(g + h)$

Quantum Computing with St

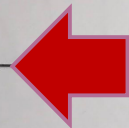
Algorithm Complexity - Matrix Multiplication

Where are we on this scale?

1-1 Comparison of running times

For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds.

	second	minute	hour	day	month	year	century
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n^2							
n^3							
2^n							
$n!$							



complexity of Strassen Algorithm is $O(n^{\lg 7})$ or ~ 2.807

Fastest!

Fast!

Linear!

Merge Sort->

Most Sorts->

Slow!

Super slow!

Intractable!

Wikipedia - Matrix Multiplication Algos

https://en.wikipedia.org/wiki/Matrix_multiplication_algorithm

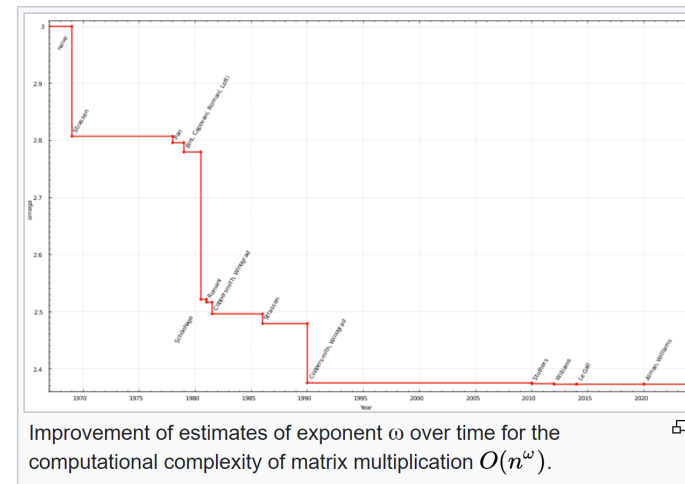
Sub-cubic algorithms [\[edit\]](#)

Further information: [Computational complexity of matrix multiplication](#)

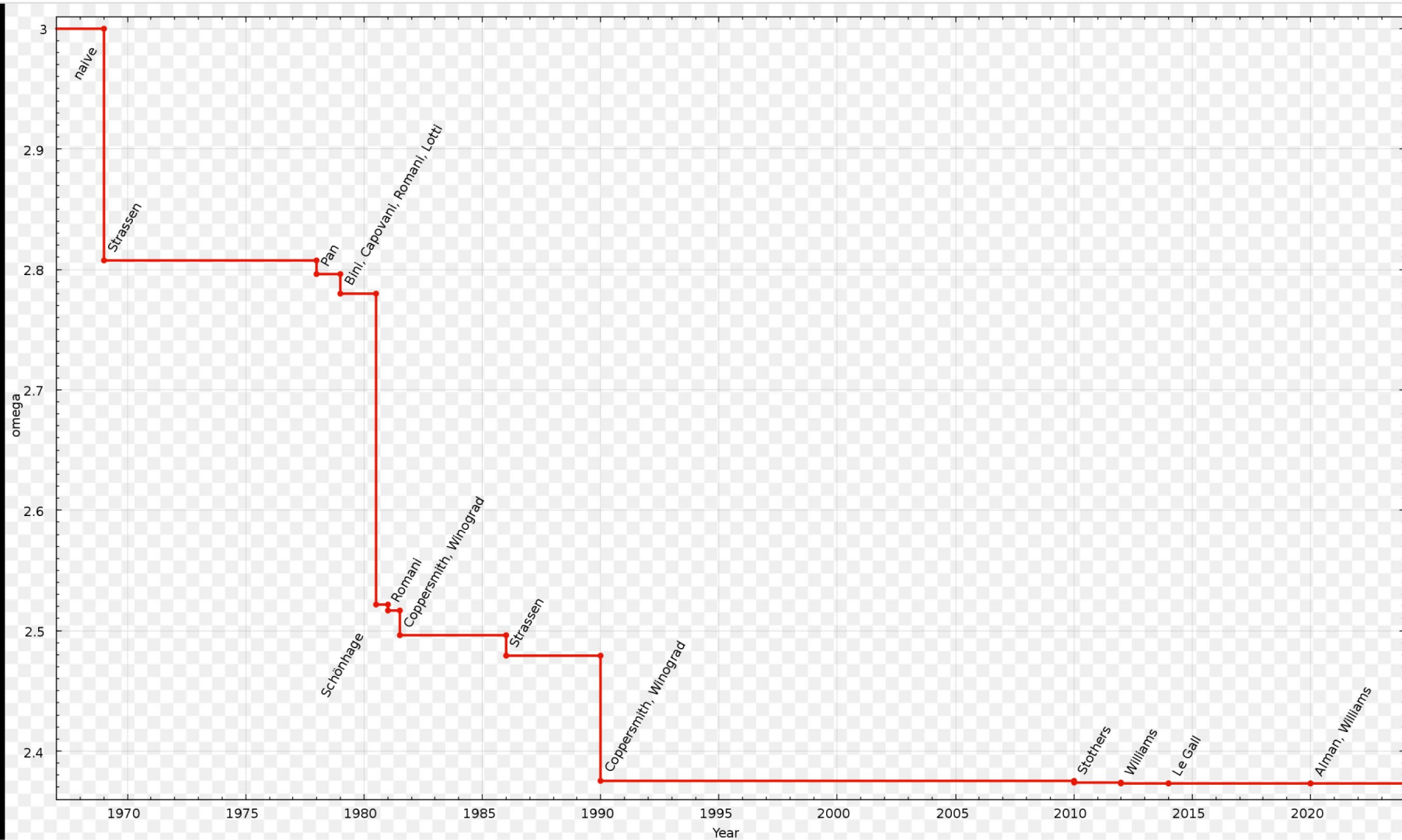
Algorithms exist that provide better running times than the straightforward ones. The first to be discovered was [Strassen's algorithm](#), devised by [Volker Strassen](#) in 1969 and often referred to as "fast matrix multiplication". It is based on a way of multiplying two 2×2 -matrices which requires only 7 multiplications (instead of the usual 8), at the expense of several additional addition and subtraction operations. Applying this recursively gives an algorithm with a multiplicative cost of $O(n^{\log_2 7}) \approx O(n^{2.807})$. Strassen's algorithm is more complex, and the [numerical stability](#) is reduced compared to the naïve algorithm,^[8] but it is faster in cases where $n > 100$ or so^[1] and appears in several libraries, such as [BLAS](#).^[9] It is very useful for large matrices over exact domains such as [finite fields](#), where numerical stability is not an issue.

It is an open question in [theoretical computer science](#) how well Strassen's algorithm can be improved in terms of [asymptotic complexity](#). The *matrix multiplication exponent*, usually denoted ω , is the smallest real number for which any $n \times n$ matrix over a field can be multiplied together using $n^{\omega+o(1)}$ field operations. The current best bound on ω is $\omega < 2.3728596$, by Josh Alman and [Virginia Vassilevska Williams](#).^[2] This algorithm, like all other recent algorithms in this line of research, is a generalization of the Coppersmith–Winograd algorithm, which was given by [Don Coppersmith](#) and [Shmuel Winograd](#) in 1990.^[10] The conceptual idea of these algorithms are similar to Strassen's algorithm: a way is devised for multiplying two $k \times k$ -matrices with fewer than k^3 multiplications, and this technique is applied recursively. However, the constant coefficient hidden by the [Big O notation](#) is so large that these algorithms are only worthwhile for matrices that are too large to handle on present-day computers.^{[11][12]}

[Freivalds' algorithm](#) is a simple [Monte Carlo algorithm](#) that, given matrices A , B and C , verifies in $\Theta(n^2)$ time if $AB = C$.



Parallel and distributed algorithms [\[edit\]](#)



Improvement of estimates of exponent ω over time for the computational complexity of matrix multiplication .

 [More details](#)

Quantum Computing - Wright St
Jochen Burghardt - Own work

CC BY-SA 4.0

Two Handle Affects between two Qubits we use “Kronecker Product” or Matrix Tensor Product

$$\begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \otimes \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix} = \begin{bmatrix} a_{1,1} \begin{bmatrix} b_{1,1} & b_{1,2} \end{bmatrix} & a_{1,2} \begin{bmatrix} b_{1,1} & b_{1,2} \end{bmatrix} \\ a_{2,1} \begin{bmatrix} b_{1,1} & b_{1,2} \end{bmatrix} & a_{2,2} \begin{bmatrix} b_{1,1} & b_{1,2} \end{bmatrix} \end{bmatrix} = \begin{bmatrix} a_{1,1}b_{1,1} & a_{1,1}b_{1,2} & a_{1,2}b_{1,1} & a_{1,2}b_{1,2} \\ a_{1,1}b_{2,1} & a_{1,1}b_{2,2} & a_{1,2}b_{2,1} & a_{1,2}b_{2,2} \\ a_{2,1}b_{1,1} & a_{2,1}b_{1,2} & a_{2,2}b_{1,1} & a_{2,2}b_{1,2} \\ a_{2,1}b_{2,1} & a_{2,1}b_{2,2} & a_{2,2}b_{2,1} & a_{2,2}b_{2,2} \end{bmatrix}.$$

Table B.2. 3-3: R_{gate} correlation matrices

1x2 case	2x2 case
$R_{\text{gate}} = R_{\text{UG}} = \begin{bmatrix} 1 & \beta \\ \beta^* & 1 \end{bmatrix}$	$R_{\text{gate}} = R_{\text{gate}} \otimes R_{\text{UG}} = \begin{bmatrix} 1 & \alpha \\ \alpha^* & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & \beta \\ \beta^* & 1 \end{bmatrix} = \begin{bmatrix} 1 & \beta & \alpha & \alpha\beta \\ \beta^* & 1 & \alpha\beta^* & \alpha \\ \alpha^* & \alpha^*\beta & 1 & \beta \\ \alpha^*\beta^* & \alpha^* & \beta^* & 1 \end{bmatrix}$

Tensor Product- Involves TWO or more QUBITS

“Matrix Tensor Product”

Results

$$\begin{pmatrix} 2 & 7 \\ 5 & 3 \end{pmatrix} \otimes \begin{pmatrix} 9 & 2 \\ 2 & 4 \end{pmatrix}$$

$\Downarrow$

$$\begin{pmatrix} \begin{pmatrix} 18 & 4 \\ 4 & 8 \end{pmatrix} & \begin{pmatrix} 63 & 14 \\ 14 & 28 \end{pmatrix} \\ \begin{pmatrix} 45 & 10 \\ 10 & 20 \end{pmatrix} & \begin{pmatrix} 27 & 6 \\ 6 & 12 \end{pmatrix} \end{pmatrix}$$

REVIEW - Week 4

Quantum Logic

Quick review: real and complex numbers

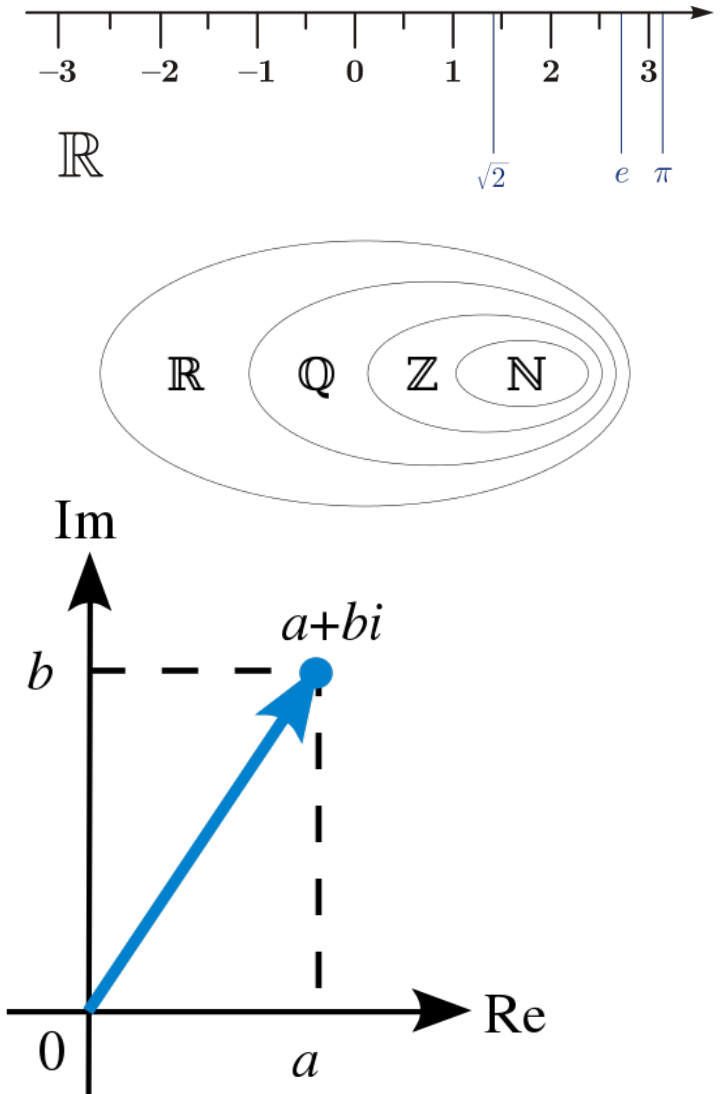
- Real numbers $\mathbb{R}$
 - Can represent distance on a line
 - Includes rational numbers, irrational numbers, natural numbers
- Complex numbers $\mathbb{C}$
 - Expressed as

$$a + bi$$

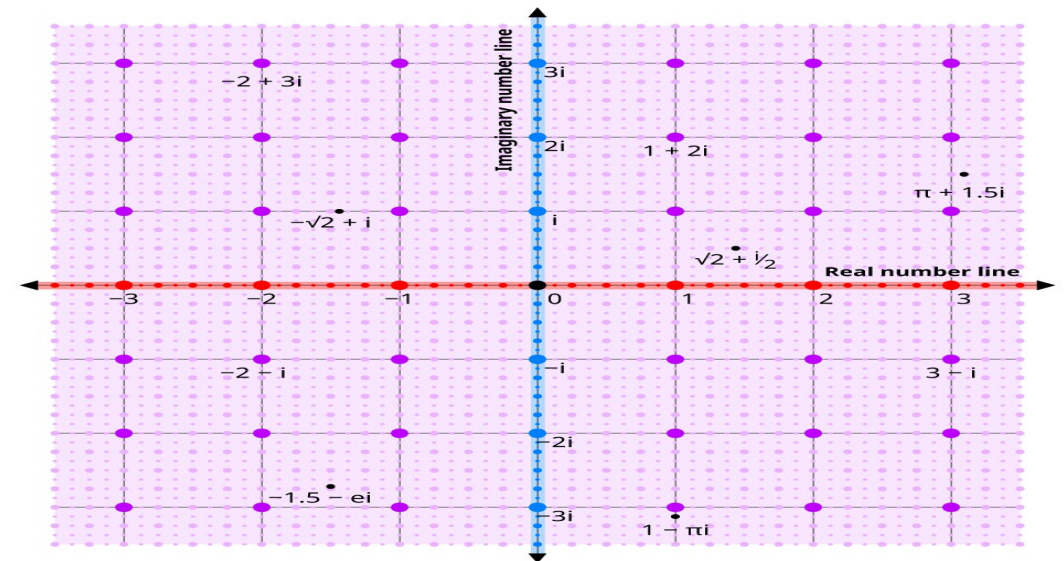
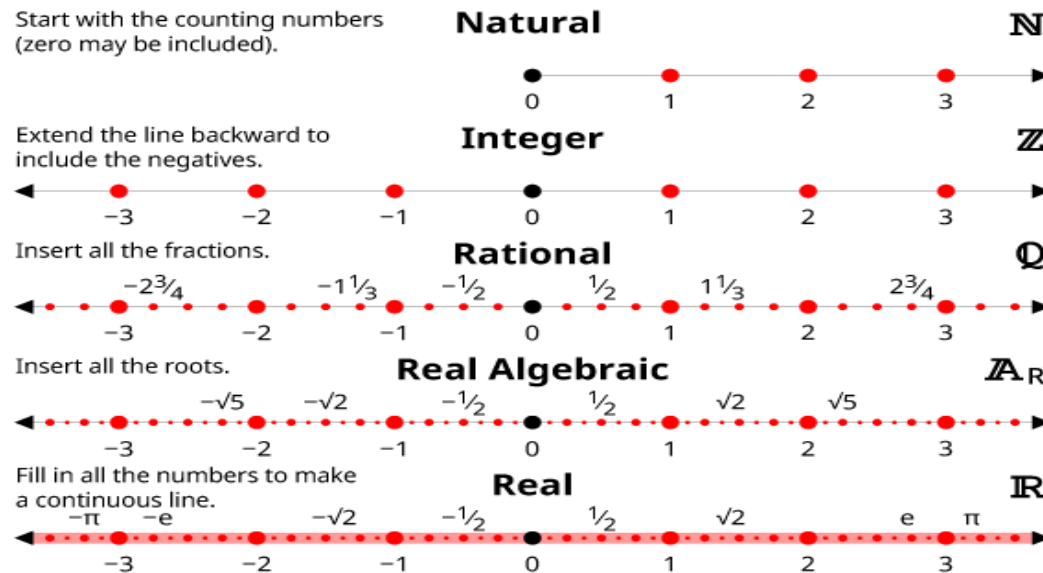
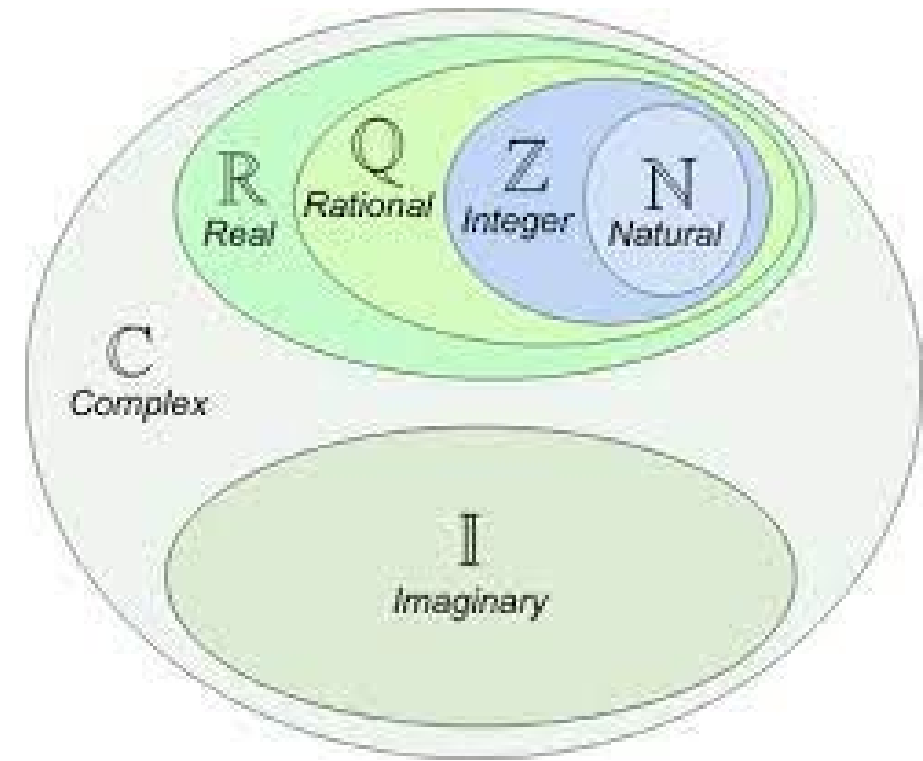
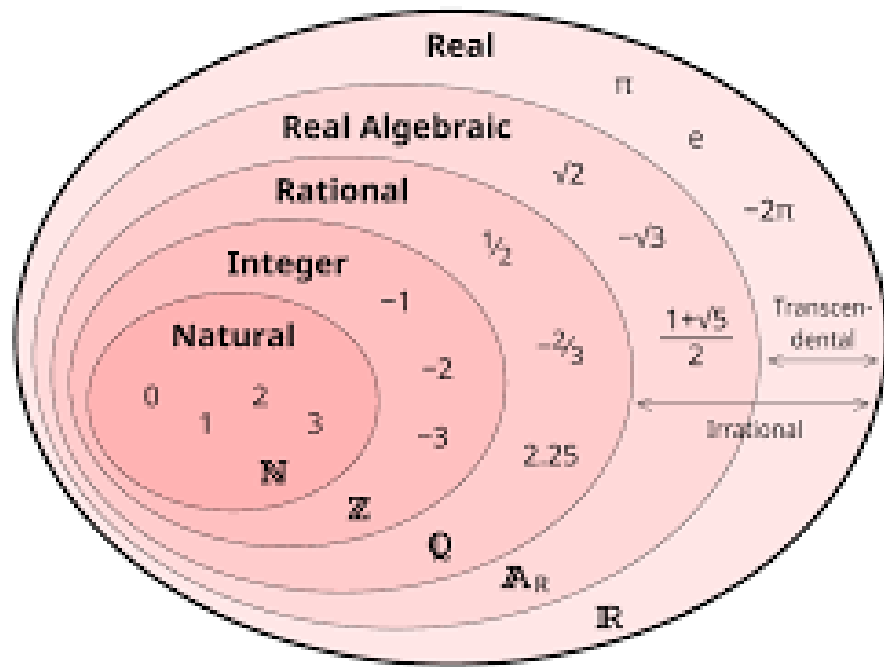
where $a, b \in \mathbb{R}$, and i is an indeterminate with $i^2 = -1$

- Complex conjugation: $(a + bi)^* = a - bi$
- Squared magnitude:

$$|a + bi|^2 = (a + bi)^*(a + bi) = a^2 + b^2$$



Number Theory



Quick review: polar form

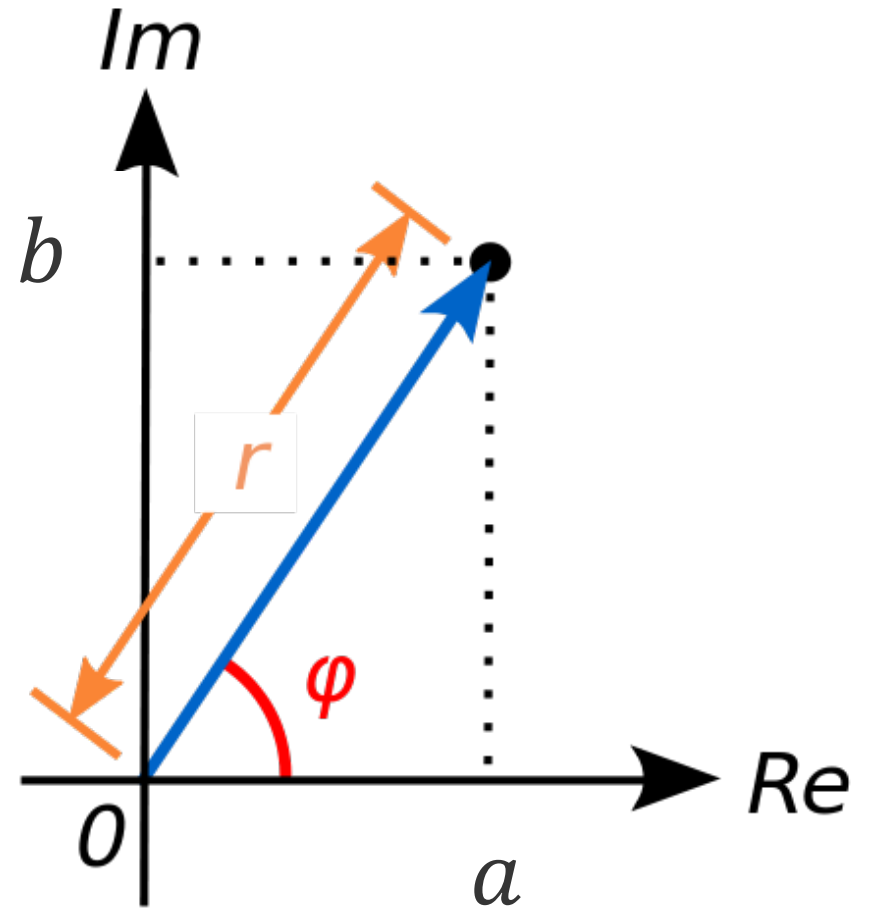
- Let $z = a + bi$

$$r = \|z\| = \sqrt{a^2 + b^2}$$

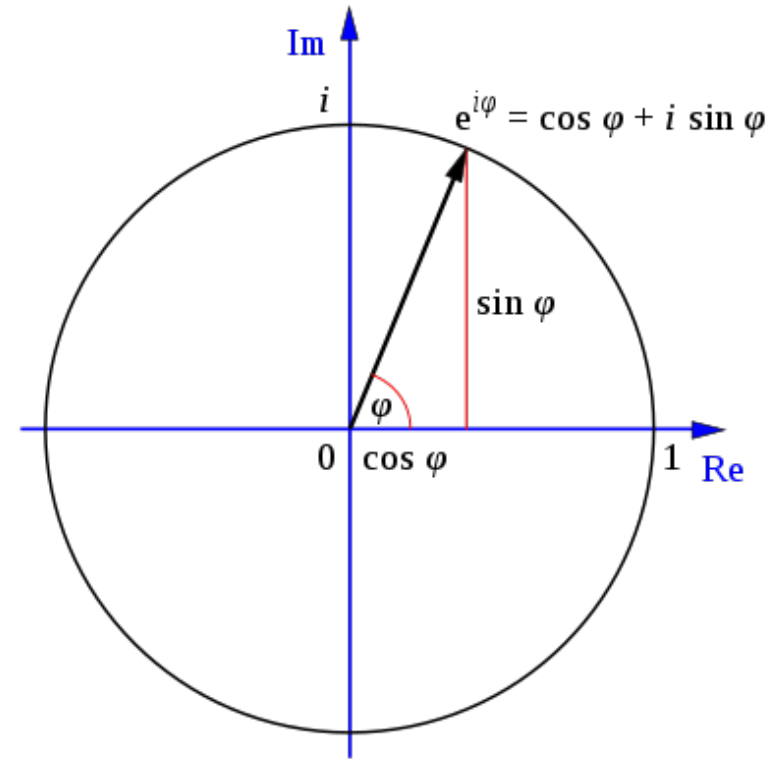
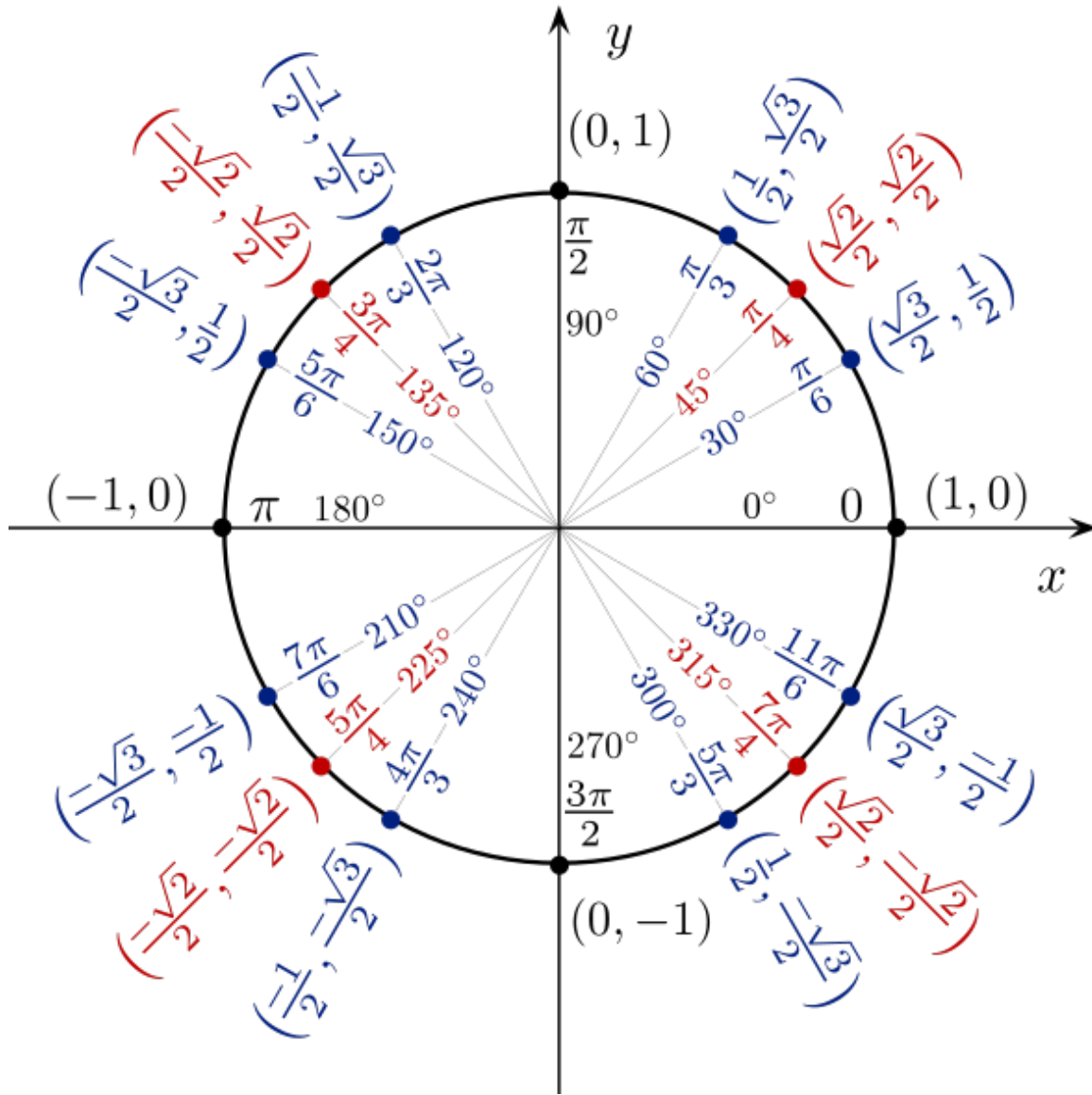
- If $b = 0$, then $r = \|a\|$

- The phase φ is called the *argument*
- (r, φ) specify point in complex plane

$$z = r(\cos\varphi + i\sin\varphi) = re^{i\varphi}$$



Quick review: trigonometric identities



$$\cos^2 \theta + \sin^2 \theta = 1$$

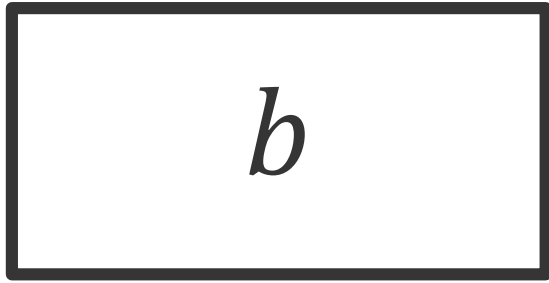
$$e^{i\theta} = \cos \theta + i \sin \theta$$

$$e^{2\pi i} = 1$$

$$e^{\pi i} = -1$$

Unit of information

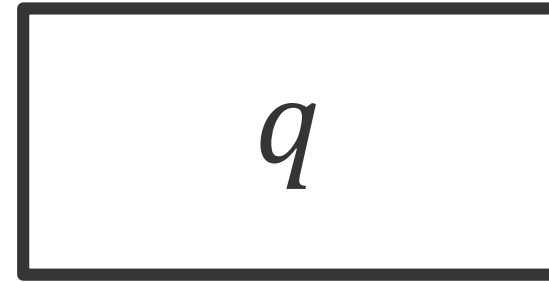
Classical: *bit*



$$b = 0 \text{ or } 1$$

$$b = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ or } \begin{pmatrix} 0 \\ 1 \end{pmatrix} - \begin{matrix} \text{"0"} \\ \text{"1"} \end{matrix}$$

Quantum: *qubit*



$$q = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ or } \begin{pmatrix} 0 \\ 1 \end{pmatrix} \text{ or } \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} - \begin{matrix} \text{"0"} \\ \text{"1"} \end{matrix}$$

$$\text{with } |c_0|^2 + |c_1|^2 = 1.$$

superposition of 0 and 1 gives quantum advantage

Unit of information

$$q$$

$$q = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ or } \begin{pmatrix} 0 \\ 1 \end{pmatrix} \text{ or } \begin{pmatrix} c_0 \\ c_1 \end{pmatrix}$$

$$\text{with } |c_0|^2 + |c_1|^2 = 1.$$

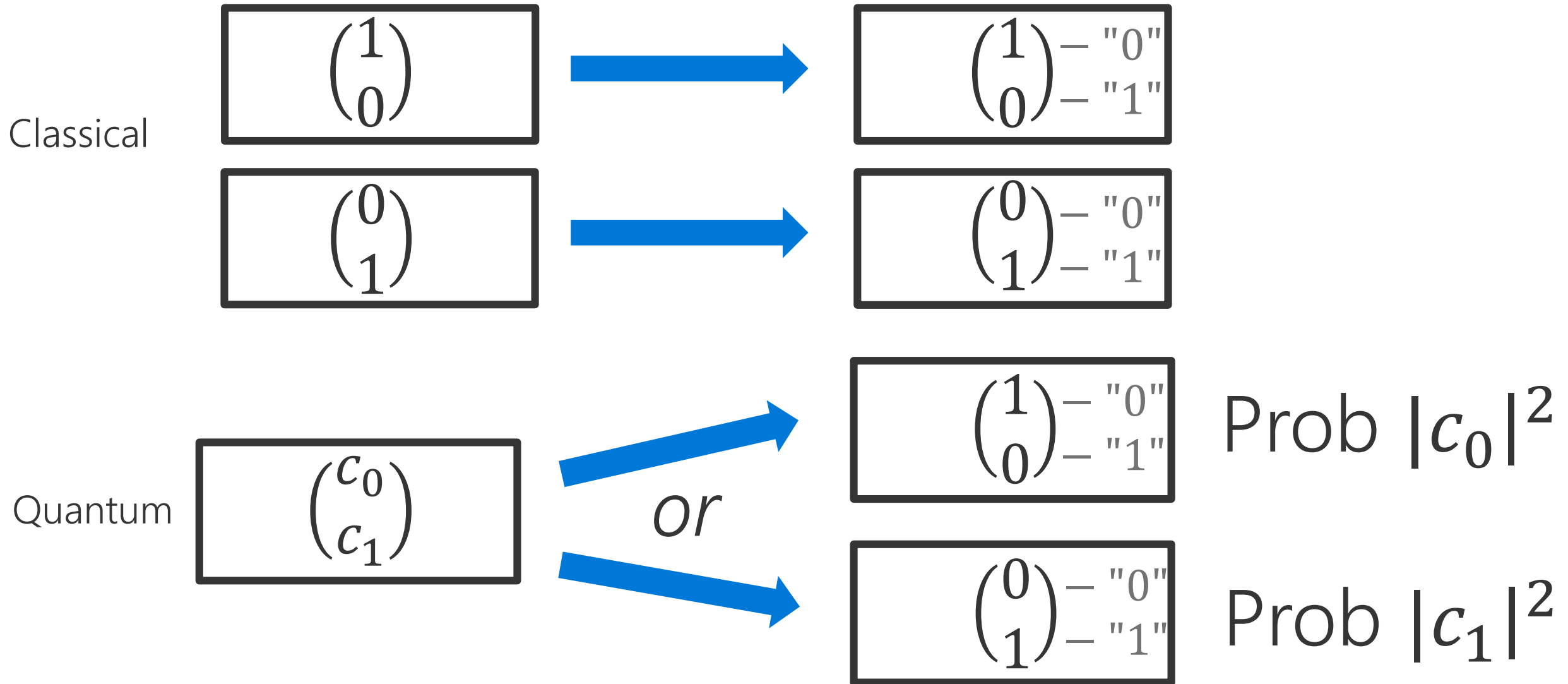
A qubit is in a
linear combination
of basis states

$$q = c_0 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + c_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} c_0 \\ c_1 \end{pmatrix}$$

Basis states "0" and "1"

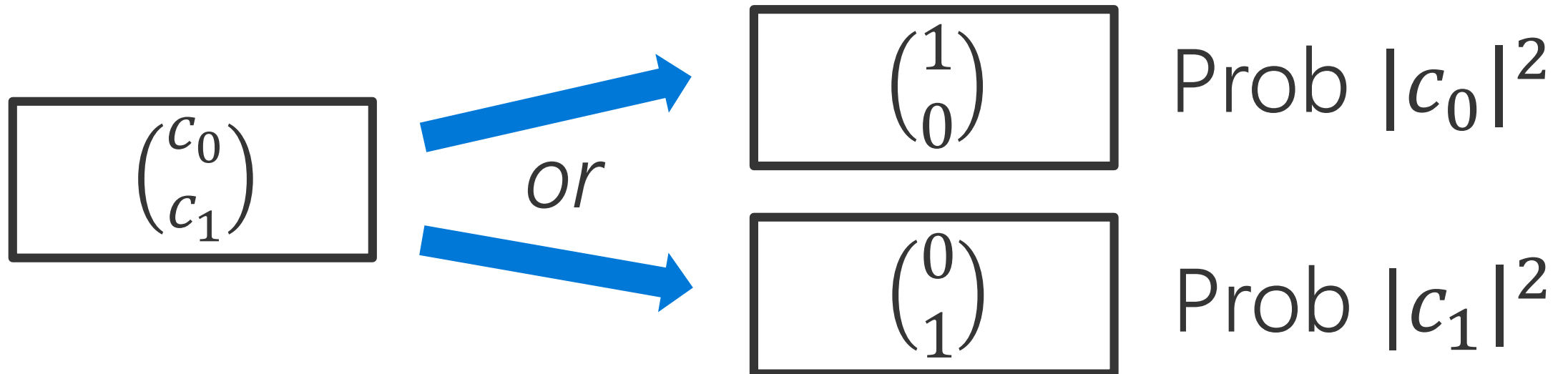
$$c_0, c_1 \text{ are complex: } \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} \in \mathbb{C}^2$$

Readout: quantum measurement



Readout: quantum measurement

- We need measurement for readout.
- Measurement *restricts power of quantum computing*, preventing us from directly learning c_0 and c_1 .
- Measurement destroys superposition.
- A qubit must be protected from unwanted measurement (known as decoherence).



Example:

$$\begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$$

This state is in
"equal superposition"

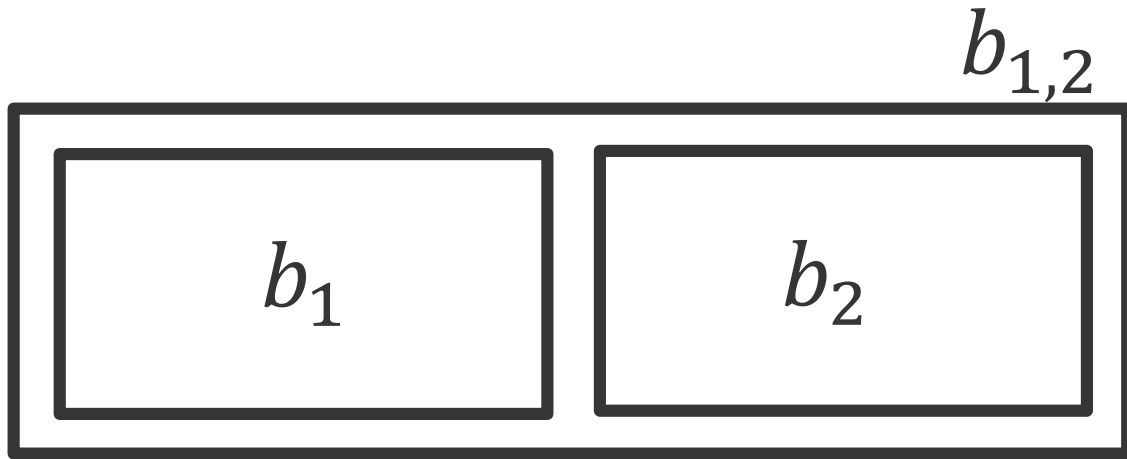
when measured

- yields "0" 50% of the time
- yields "1" 50% of the time

This state is sometimes called the "+" state.

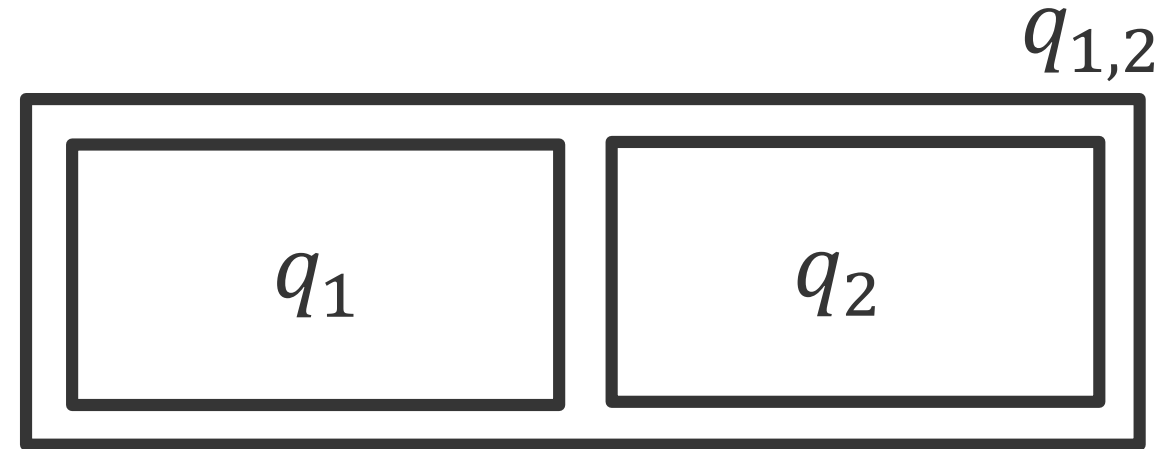
Multiple Qubits

Storing more information: multiple qubits



$b_{1,2} = 00 \text{ or } 01 \text{ or } 10 \text{ or } 11$

$$\begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \text{ or } \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \begin{matrix} \text{--- "00"} \\ \text{--- "01"} \\ \text{--- "10"} \\ \text{--- "11"} \end{matrix}$$



$$q_{1,2} = \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix} \begin{matrix} \text{--- "00"} \\ \text{--- "01"} \\ \text{--- "10"} \\ \text{--- "11"} \end{matrix}$$

$$|c_{00}|^2 + |c_{01}|^2 + |c_{10}|^2 + |c_{11}|^2 = 1.$$

Measure "00" with prob $|c_{00}|^2$, etc.

Reading out more information: multiple qubits

$$q_{1,2} = \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix} \begin{matrix} \text{-- "00"} \\ \text{-- "01"} \\ \text{-- "10"} \\ \text{-- "11"} \end{matrix} \quad q_{1,2} = c_{00} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} + c_{01} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} + c_{10} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} + c_{11} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

$$|c_{00}|^2 + |c_{01}|^2 + |c_{10}|^2 + |c_{11}|^2 = 1.$$

Measure "00" with prob $|c_{00}|^2$, etc.

Measure first qubit to be "0" with prob $|c_{00}|^2 + |c_{01}|^2$, with post-measurement state

$$q_1 = \frac{1}{\sqrt{|c_{00}|^2 + |c_{01}|^2}} \left(c_{00} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} + c_{01} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \right)$$

Quick review: Dirac notation

"Ket" notation: $|\cdot\rangle$ denotes a column vector

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, |q\rangle = \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = c_0|0\rangle + c_1|1\rangle$$

Let $|\psi\rangle$ denote a column vector, where $c_i \in \mathbb{C}^{2^n}$:

$$|\psi\rangle = \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{2^n-2} \\ c_{2^n-1} \end{pmatrix} = \sum_{i=0}^{2^n-1} c_i |i\rangle$$

We will sometimes denote the i th basis state in integer form for compactness, rather than in binary

n is the number of qubits in the quantum state.

Quick review: Dirac notation

$$|\psi\rangle = \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix} \begin{matrix} \text{-- "00"} \\ \text{-- "01"} \\ \text{-- "10"} \\ \text{-- "11"} \end{matrix} = c_{00} |00\rangle + c_{01} |01\rangle + c_{10} |10\rangle + c_{11} |11\rangle \\ = c_0 |0\rangle + c_1 |1\rangle + c_2 |2\rangle + c_3 |3\rangle$$

Ket Notation:

Shorthand notation for column vectors.

Used a lot in the literature.

Sometimes more compact.

$$|\varphi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} |000\rangle + \frac{1}{\sqrt{2}} |110\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |6\rangle)$$

Example: Bell states

Can the following state be represented as a product state?

$$\frac{1}{\sqrt{2}} |00\rangle + \frac{1}{\sqrt{2}} |11\rangle$$

No!

This is an entangled state. It cannot be separated into a product of two states.

This state can exhibit “nonlocal” correlations.

Entanglement

Unentangled

Alice

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Bob

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)(|0\rangle + |1\rangle)$$

$$q_{AB} = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \begin{matrix} \text{-- "00"} \\ \text{-- "01"} \\ \text{-- "10"} \\ \text{-- "11"} \end{matrix}$$

$$\Pr(00) = \Pr(01) = \Pr(10) = \Pr(11) = \frac{1}{4}.$$

Entangled

Alice

Bob

cannot write separately

$$\frac{1}{\sqrt{2}} |00\rangle + \frac{1}{\sqrt{2}} |11\rangle$$

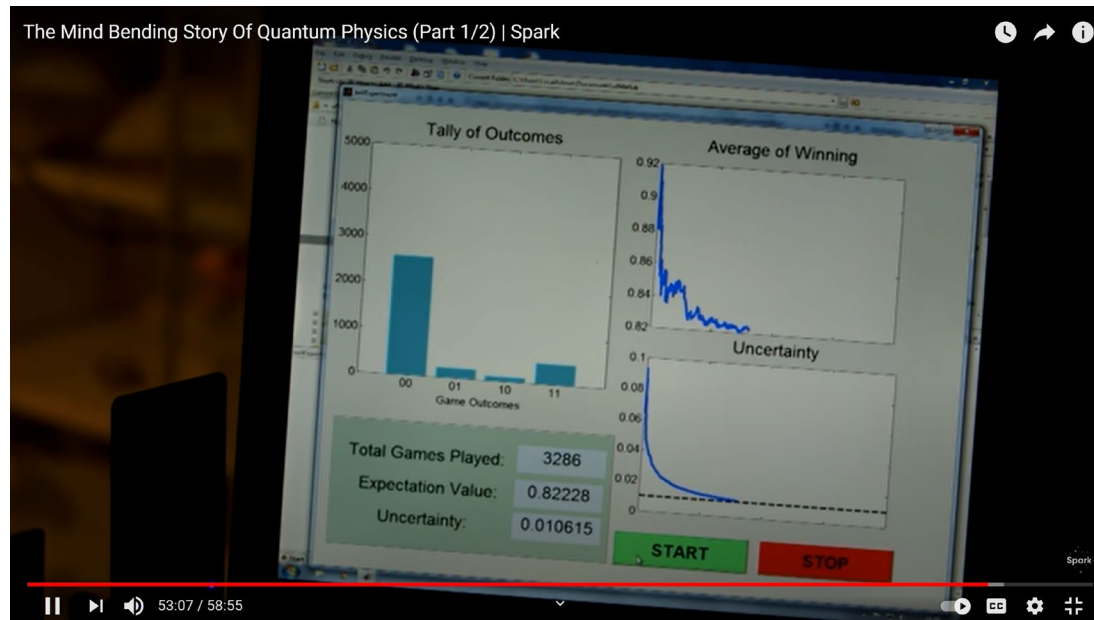
$$q_{AB} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} \begin{matrix} \text{-- "00"} \\ \text{-- "01"} \\ \text{-- "10"} \\ \text{-- "11"} \end{matrix}$$

$$\Pr(00) = \Pr(11) = \frac{1}{2}.$$

$$\Pr(01) = \Pr(10) = 0.$$

What's a Bell State?

- Watch more of the Al Khalili Documentary
- Einstein's Nightmare
- 33:33 to 54:20
- <https://www.youtube.com/watch?v=ISdBaf-ysl0&t=2717s>



REVIEW - Week 5

Quantum Gates

Basics for quantum computing

- ▶ Initialize the quantum state to $|0\rangle$
 - ▶ State preparation
- ▶ Compute on the quantum states
 - ▶ Quantum operations or “gates”
 - ▶ State evolution
- ▶ Readout the information
 - ▶ Measurement

**We sometimes call this the
GROUND STATE.**

**Or think of it as the
Starting State.**

Steps in a Quantum Algorithm

A quantum algorithm uses several qubits which together constitute a *quantum register*. From the viewpoint of a single qubit, its activity looks like

1. I, the qubit, am in initial state $|0\rangle$.
2. I get moved into a standard superposition. Don't peek!
3. Steps in the algorithm apply zero or more reversible operations to me.
4. I get measured and always become $|0\rangle$ or $|1\rangle$.
5. When these get read out for classical use, they are converted to 0 or 1.

It's what happens in steps 2 and 3 that make quantum computing interesting. For the unit sphere, which is called the *Bloch sphere* in quantum computing, step 2 means I get moved to a designated point/state on the equator. Step 3 translates to getting moved to other points/states. This movement is a result of what the algorithm is doing and causing to happen.

The final two steps are like telling the qubit “look, you can't stay somewhere in superposition forever. You need to decide whether you will be 0 or 1. This is not optional.” Where it is in superposition, in what state on the sphere, determines the relative probabilities of the qubit ultimately yielding a classical bit value of 0 or 1.

C² and Measurement

From Dancing With Qubits: Chapter 7 - page 229

We explore and return to this statement throughout the rest of this chapter:

A *qubit*—a *quantum bit*—is the fundamental unit of quantum information. At any given time, it is in a superposition state represented by a linear combination of vectors $|0\rangle$ and $|1\rangle$ in $\mathbb{C}^2$:

$$a|0\rangle + b|1\rangle \quad \text{where} \quad |a|^2 + |b|^2 = 1 .$$

Through *measurement*, a qubit is forced to collapse irreversibly through projection to either $|0\rangle$ or $|1\rangle$. The probability of its doing either is $|a|^2$ and $|b|^2$, respectively. a and b are called *probability amplitudes*.

If necessary, we can convert (“read out”) $|0\rangle$ and $|1\rangle$ to classical bit values of 0 and 1.

Quantum State Representation

From Dancing With Qubits: Chapter 7 - page 235

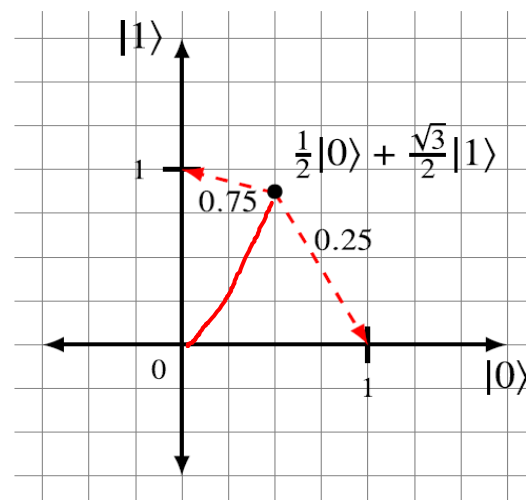
7.3.1 Quantum state representation

If $|\psi\rangle = a|0\rangle + b|1\rangle$ is a quantum state, we know $|\psi|^2 = |a|^2 + |b|^2 = 1$. This is the same as saying $\langle\psi|\psi\rangle = 1$.

The values $|a|^2$ and $|b|^2$ are non-negative numbers in $\mathbb{R}$ that are the probabilities that $|\psi\rangle$ will go to $|0\rangle$ or $|1\rangle$, respectively, when measured.

Consider the following example in which, for illustrative purposes, I have made some simplifications and taken some labeling liberties. Suppose a and b can only be real.

Let $a = \frac{1}{2}$ and $b = \frac{\sqrt{3}}{2}$. Then $|a|^2 = \left|\frac{1}{2}\right|^2 = 0.25$ and $|b|^2 = \left|\frac{\sqrt{3}}{2}\right|^2 = 0.75$. This means that the state of the qubit has a 25% chance of collapsing to $|0\rangle$ when measured and a 75% chance of collapsing to $|1\rangle$.



Bloch Sphere

From Dancing With Qubits: Chapter 7 - page 248

7.5 The Bloch sphere

We describe the state of a qubit by a vector

$$a|0\rangle + b|1\rangle = r_1 e^{i\varphi_1} |0\rangle + r_2 e^{i\varphi_2} |1\rangle$$

in $\mathbb{C}^2$ with r_1 and r_2 non-negative numbers in $\mathbb{R}$.

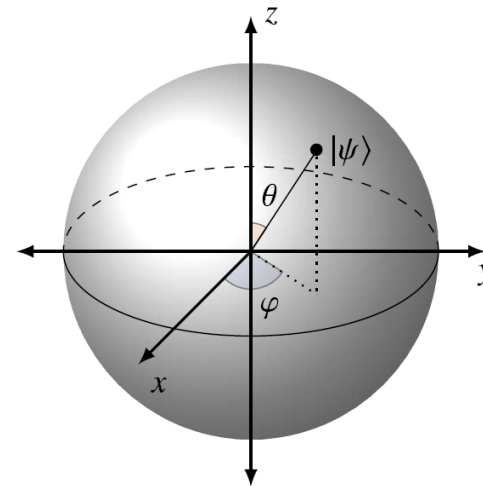
The magnitudes r_1 and r_2 are related by $r_1^2 + r_2^2 = 1$. This is a mathematical condition. We saw in [section 7.3](#) that it's the relative phase of $\varphi_2 - \varphi_1$ that is significant and not the individual phases φ_1 and φ_2 . This is a physical condition and it also means we can take a to be real.

We also saw that we could represent a quantum state as

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + \sin\left(\frac{\theta}{2}\right) e^{i\varphi} |1\rangle.$$

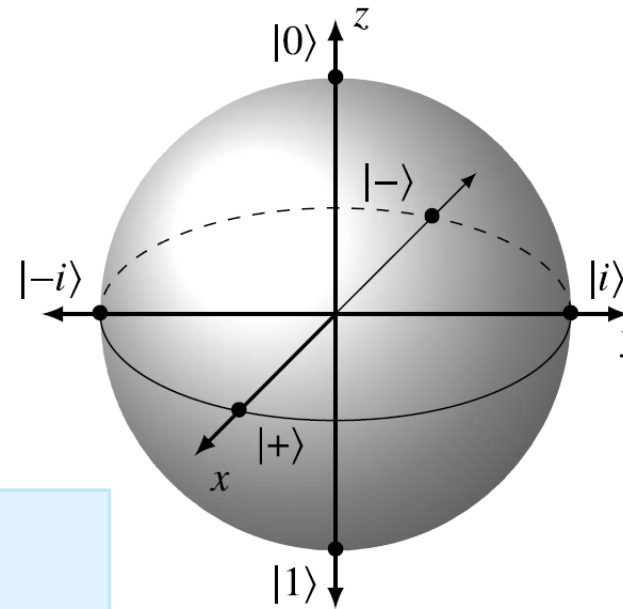
We do this via a non-linear projection and a change of coordinates, and get a point on the surface of the *Bloch sphere*.

The two angles have the ranges $0 \leq \theta \leq \pi$ and $0 \leq \varphi < 2\pi$. θ is measured from the positive z axis and φ from the positive x axis in the xy -plane.



Bloch Sphere (Different Explanation)

From Dancing With Qubits: Chapter 7 - page 252



We have now seen three pairs of basis elements and they go by different names:

Basis	Common name	Bloch sphere axis name
$\{ 0\rangle, 1\rangle\}$	Computational	Z
$\{ +\rangle, -\rangle\}$	Hadamard	X
$\{ i\rangle, -i\rangle\}$	Circular	Y

The Y basis is the same as the circular basis. In this case we also have the alternative notation $|i\rangle = |\uparrow\rangle$ and $|-i\rangle = |\downarrow\rangle$.

Unitary Matrix

From Dancing With Qubits: Chapter 5 - page 184

5.7.5 Unitary transformations

What are the characteristics of linear transformations that preserve length? That is, if $L : V \rightarrow V$ and it is always true that $\|L(\mathbf{v})\| = \|\mathbf{v}\|$, what can we say about the matrix of L ?

If U is a complex square matrix then it is *unitary* if its adjoint $U^\dagger$ is also its inverse U^{-1} . Hence $UU^\dagger = U^\dagger U = I$. This means the columns of U are orthonormal, as are the rows.

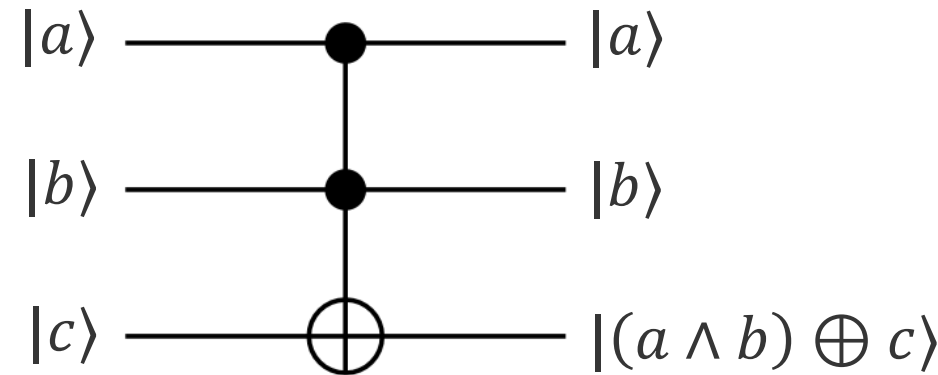
We call this symbol “Dagger” or the Adjoint

The matrices corresponding to gates/operations on qubits are all unitary.

Quantum logic gates

- How do we make AND and OR gates reversibly?
- We use a Toffoli gate.
- Toffoli (**CCX**) negates the third qubit iff the first two qubits are both $|1\rangle$:
 $|a, b, c\rangle \rightarrow |a, b, (a \wedge b) \oplus c\rangle$

XOR



$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Changing the phase

- How do we modify phase?

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$
$$Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle$$

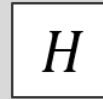
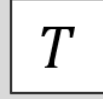
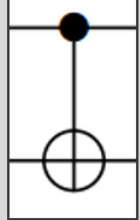
$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$
$$S(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle + i\beta|1\rangle$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$
$$Y(\alpha|0\rangle + \beta|1\rangle) = -i\beta|0\rangle + i\alpha|1\rangle$$

$$I = -iXYZ$$

$$XZ = -ZX$$

Quantum gates

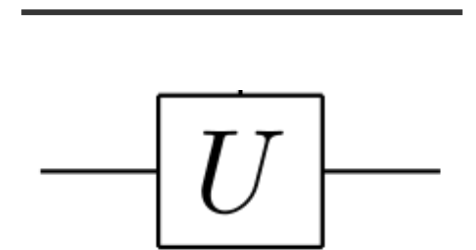
Gate	Matrix	Symbol
Pauli X	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	
Pauli Z	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	
Pauli Y	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	
Phase S	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$	
Hadamard H	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	
T = " $\pi/8$ "	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	
Controlled Not CNOT	$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	

Example:

$$U = \begin{pmatrix} u_0 & u_1 \\ u_2 & u_3 \end{pmatrix}$$

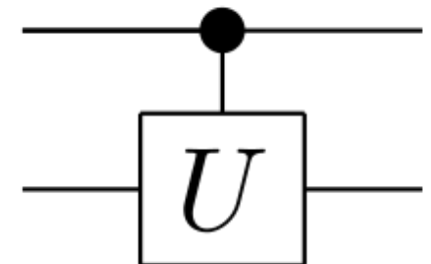
Apply U to one of two qubits in a system. I is applied to the other qubit.

$$I \otimes U = \begin{pmatrix} u_0 & u_1 & 0 & 0 \\ u_2 & u_3 & 0 & 0 \\ 0 & 0 & u_0 & u_1 \\ 0 & 0 & u_2 & u_3 \end{pmatrix}$$



This is different than controlling the application of U based on a qubit:

$$CU = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_0 & u_1 \\ 0 & 0 & u_2 & u_3 \end{pmatrix}$$



IBM Quantum Composer

- <https://quantum-computing.ibm.com/composer>

The screenshot displays the IBM Quantum Composer web application. The interface is divided into several sections:

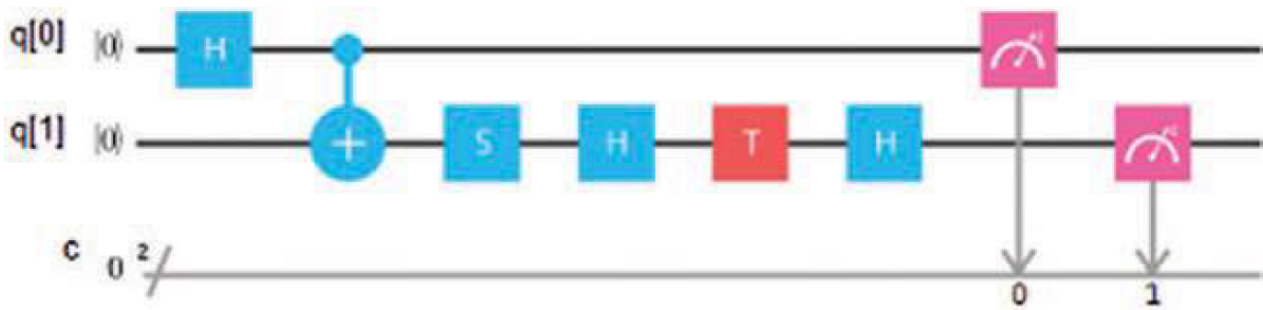
- Composer files:** A sidebar on the left showing a list of files. The current file is "Untitled circuit", updated "a few seconds ago".
- File menu:** A top menu bar with options: File, Edit, Inspect, View, Share, and a "Setup and run" button.
- Quantum Circuit Editor:** The central workspace for building a quantum circuit. It includes a toolbar with gates (H, CNOT, T, S, Z, T†, S†, P, RZ, etc.), a list of qubits (q[0], q[1], c[2]), and a visual representation of the circuit.
- Visualizations:** Two panels at the bottom. The left panel shows "Probabilities" as a bar chart for computational basis states (00, 01, 10, 11). The right panel shows a "Q-sphere" visualization of the quantum state.
- Code Editor:** A panel on the right showing the OpenQASM 2.0 code for the circuit.

```
1 OPENQASM 2.0;  
2 include "qelib1.inc";  
3  
4 qreg q[2];  
5 creg c[2];  
6  
7 x q[0];
```

The bottom of the image shows a Windows taskbar with the system clock indicating 1:39 PM on 9/14/2021.

Probabilities - Not Perfect Answers

Table 3-4. Quantum Circuits for Bell States

Bell state measurement		Result for 100 shots	
AB (ZW)			
		c[2]	Probability
		11	0.39
		10	0.06
		00	0.46
		01	0.09